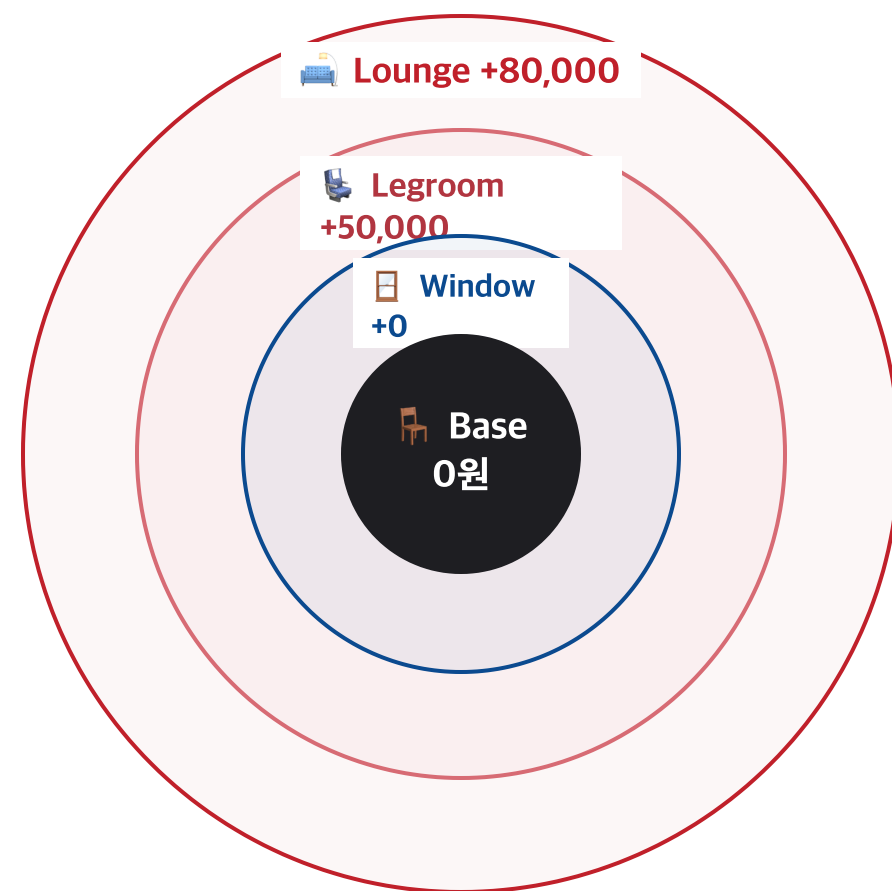


📦 감싸서 쌓아 올린다

좌석 부가옵션의 무한한 조합을, 클래스 폭발 없이 런타임에 누적하는 법

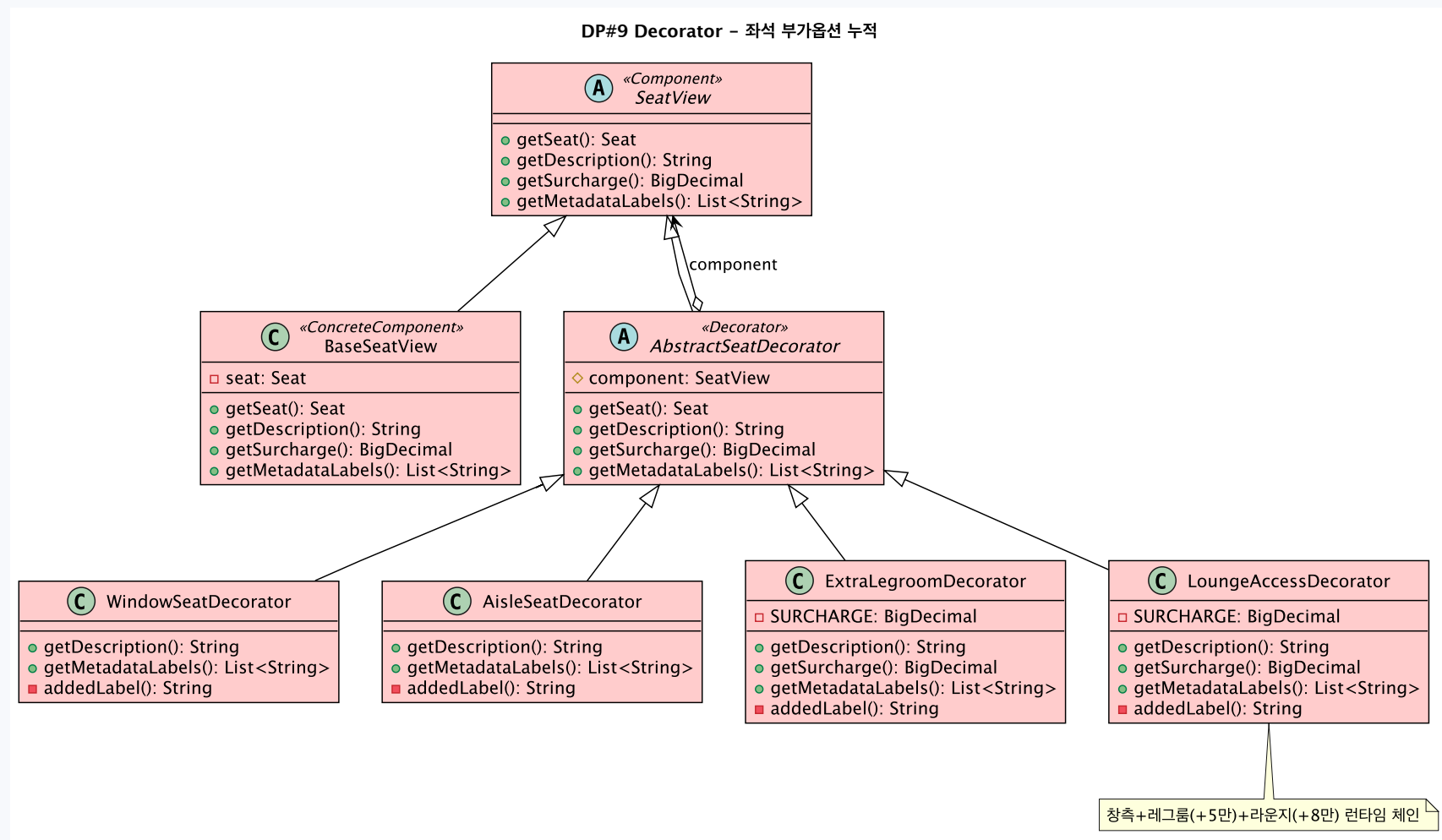


옵션을 **상속**으로 만들면 조합마다 클래스가 필요해 **2ⁿ로 폭발**합니다. 그래서 Decorator는 발상을 완전히 바꿉니다.

- 📦 각 옵션을 독립된 **객체(데코레이터)** 하나로 만듭니다. 그래서 옵션이 많아져도 조합끼리 곱해지지 않습니다.
- 🍎 그 객체로 기본 좌석을 바깥에서 **한 겹씩 감쌉니다**. 새 조합이 필요하면 객체를 더 돌려 주기만 하면 됩니다.
- ♻️ 감싼 결과도 **같은 SeatView 타입**이라 새 옵션은 클래스 하나만 추가하면 되고, 기존 코드는 그대로입니다(OCP).

Decorator의 네 가지 역할

교과서 역할이 우리 클래스로 이렇게 나뉜다



 **Component 역할은 SeatView가** 말합니다. 기본 좌석과 모든 옵션이 똑같이 따르는 공통 추상 타입으로, 네 가지 연산만 약속해 둡니다.

 **ConcreteComponent은 BaseSeatView**입니다. 진짜 좌석을 감싼 기본 좌석으로, 쌓인 구조에서 언제나 가장 안쪽에 놓입니다.

 **Decorator는 AbstractSeatDecorator**입니다. 감싼 안쪽 좌석을 들고, 모든 요청을 일단 안쪽으로 넘기는(forward) 포장지의 뼈대입니다.

 **ConcreteDecorator는 Window·Aisle·ExtraLegroom·Lounge(A~D)**입니다. 각자 자기 라벨이나 요금을 안쪽 결과 위에 더하는 실제 옵션들입니다.

① COMPONENT

SeatView

좌석을 4가지 관점으로 보는 공통 약속

SeatView.java · Component · abstract

// ▶ 좌석을 보는 4가지 '관점'만 약속한다 (구현은 비움)

```
public abstract class SeatView {  
  
    public abstract Seat getSeat();           // 좌석 본체  
    public abstract String getDescription();  // 화면 표시 문구  
    public abstract BigDecimal getSurcharge(); // 추가 요금  
    public abstract List<String> getMetadataLabels(); // 태그 목록  
}
```



이 추상 클래스는 패턴 전체가 따르는 공통 약속입니다. 좌석을 본체·문구·요금·태그라는 **네 가지 관점으로만** 보겠다고 선언하고, 구현은 비워 둡니다.



기본 좌석과 옵션 포장지가 **둘 다 이 SeatView를 상속**합니다. 같은 타입이라 포장지가 기본 좌석을, 또 다른 포장지를 자유롭게 감쌀 수 있습니다.



그래서 화면 코드는 이 네 메서드만 알면 되고, 안에 옵션이 몇 겹 들었는지는 **전혀 신경 쓰지 않아도** 됩니다.

② CONCRETE COMPONENT

BaseSeatView

옵션이 하나도 안 붙은, 가장 기본이 되는 좌석

BaseSeatView.java · ConcreteComponent

```
public class BaseSeatView extends SeatView {

    // ▶ 진짜 좌석 하나를 감쌌 (체인의 맨 안쪽)
    private final Seat seat;

    public BaseSeatView(Seat seat) {
        this.seat = Objects.requireNonNull(seat);
    }

    @Override public Seat getSeat() { return seat; }

    // ▶ "12A (BUSINESS)" 같은 기본 정보만 보여 줌
    @Override public String getDescription() {
        return seat.getSeatNumber() + " (" + seat.getCabinClass() + ")";
    }

    // ▶ 추가요금 0원 · 태그 빈 목록에서 출발 (바깥이 여기에 누적)
    @Override public BigDecimal getSurcharge() { return BigDecimal.ZERO; }
    @Override public List<String> getMetadataLabels() { return new ArrayList<>(); }
}
```

 BaseSeatView · 12A (BUSINESS) · 요금 0원 · 태그 []

-  BaseSeatView는 아직 아무 옵션도 안 붙은 '맨 처음 좌석'입니다. 진짜 Seat 객체 하나를 받아 '12A, 비즈니스석' 정도의 기본 정보만 보여 줍니다.
-  옵션이 없으니 추가 요금은 0원, 태그 목록은 빈 리스트에서 출발합니다. 바깥 데코레이터들이 이 0원과 빈 리스트 위에 자기 값을 더해 갑니다.
-  모든 감싸기가 이 좌석에서 출발하기 때문에, BaseSeatView는 여러 겹으로 쌓인 좌석의 가장 안쪽에 놓입니다.

③ DECORATOR

AbstractSeatDecorator

옵션 포장지들의 뼈대 — 일단 안쪽에 넘긴다

AbstractSeatDecorator.java · Decorator · abstract

```
public abstract class AbstractSeatDecorator extends SeatView {

    // ▶ ← 내가 감싼 안쪽 좌석 (자기참조 = 무한 중첩의 비결)
    protected final SeatView component;

    protected AbstractSeatDecorator(SeatView component) {
        this.component = Objects.requireNonNull(component);
    }

    // ▶ 아래 네 메서드 모두 'forward' — 그냥 안쪽 component 에 넘긴다
    @Override public Seat getSeat() { return component.getSeat(); }
    @Override public String getDescription() { return component.getDescription(); }
    @Override public BigDecimal getSurcharge() { return component.getSurcharge(); }
    @Override public List<String> getMetadataLabels() {
        return new ArrayList<>(component.getMetadataLabels());
    }
}
```

이 포장지 — component → 안쪽 SeatView



이 클래스가 Decorator 패턴의 **심장**입니다. 자기도 SeatView이면서, 생성자에서 받은 **또 다른 SeatView**를 **안에 품습니다** — 그게 자기가 감싼 '안쪽 좌석'입니다.



기본 동작은 모든 연산을 **안쪽에게 그대로 물어보고 그대로 돌려주는 것**(forward)입니다. 위 네 메서드 모두 component로 넘기기만 합니다.



실제 옵션들은 이 뼈대를 물려받아 필요한 메서드만 고칩니다. 그 방식은 항상 「**super로 받고 → 내 것 더하고 → 돌려준다**」입니다.

WindowSeatDecorator

창가 표시 · 요금 0원

WindowSeatDecorator.java · ConcreteDecorator A

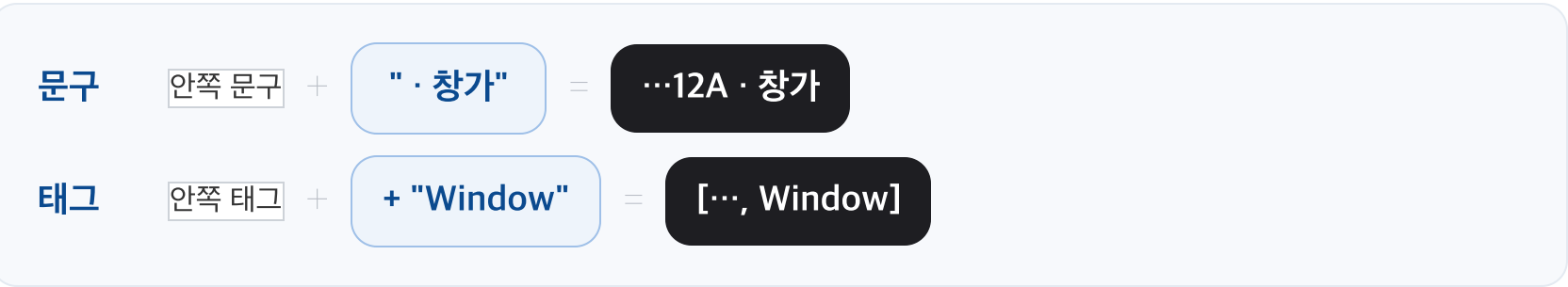
```
public class WindowSeatDecorator extends AbstractSeatDecorator {

    public WindowSeatDecorator(SeatView component) { super(component); }

    // ▶ 안쪽 문구를 받아 '· 창가' 를 덧붙인다
    @Override public String getDescription() {
        return super.getDescription() + addedBehavior();
    }

    // ▶ 안쪽 태그를 받아 "Window" 하나 추가
    @Override public List<String> getMetadataLabels() {
        List<String> labels = super.getMetadataLabels();
        labels.add("Window");
        return labels;
    }

    // ▶ getSurchage 가 없음 → 안쪽 요금(0)이 그대로 통과 = 창가는 0원
    private String addedBehavior() { return " · 창가"; }
}
```



-  창가는 화면에 표시만 하고 **요금**은 **안 붙는** 옵션이라, getDescription과 getMetadataLabels만 고쳐 씁니다.
- +** 문구는 super로 안쪽 것을 받아 '· 창가'를 이어 붙이고, 태그 목록에는 안쪽 것을 받아 'Window'를 하나 더 넣습니다.
-  **getSurchage**를 **override**하지 **않은 것**이 포인트입니다. 부모의 forward가 작동해 안쪽 요금 0원이 그대로 통과합니다.



AisleSeatDecorator

통로 표시 · 요금 0원

AisleSeatDecorator.java · ConcreteDecorator B

```
public class AisleSeatDecorator extends AbstractSeatDecorator {  
  
    public AisleSeatDecorator(SeatView component) { super(component); }  
  
    // ▶ 안쪽 문구 + '· 통로'  
    @Override public String getDescription() {  
        return super.getDescription() + addedBehavior();  
    }  
  
    // ▶ 태그에 "Aisle" 추가  
    @Override public List<String> getMetadataLabels() {  
        List<String> labels = super.getMetadataLabels();  
        labels.add("Aisle");  
        return labels;  
    }  
  
    // ▶ 여기도 getSurcharge 없음 → 통로도 요금 0원  
    private String addedBehavior() { return " · 통로"; }  
}
```



-  통로 옵션은 창가 옵션과 구조가 완전히 똑같습니다 — 문구에 ' · 통로'를, 태그에 'Aisle'을 더하고, 요금은 건드리지 않아 0원이 그대로 통과합니다.
-  이렇게 같은 모양이 반복된다는 게 장점입니다. 새 '라벨 옵션'이 필요하면 이 클래스를 그대로 베껴 단어만 바꾸면 됩니다.
- 1

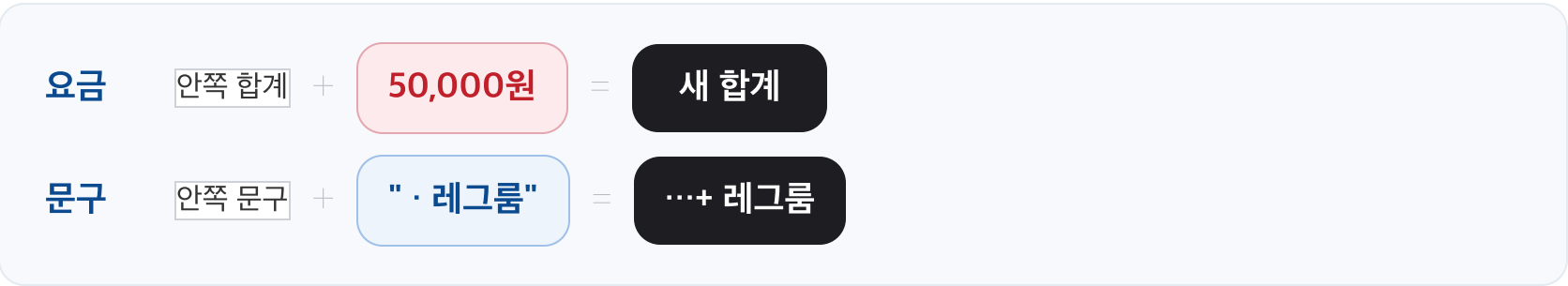
 한 좌석은 창가이거나 통로이거나 둘 중 하나이므로, 조립을 담당하는 Client는 Window와 Aisle 중 **하나만** 씁습니다.

ExtraLegroomDecorator

프리미엄 레그룸 · 요금 +50,000원

ExtraLegroomDecorator.java · ConcreteDecorator C

```
public class ExtraLegroomDecorator extends AbstractSeatDecorator {  
  
    // ▶ 이 옵션이 더하는 추가 요금  
    private static final BigDecimal SURCHARGE = new BigDecimal("50000");  
  
    public ExtraLegroomDecorator(SeatView component) { super(component); }  
  
    @Override public String getDescription() {  
        return super.getDescription() + addedBehavior();  
    }  
  
    // ▶ ★ 안쪽 합계 + 내 요금(50,000) = 요금 누적의 핵심  
    @Override public BigDecimal getSurcharge() {  
        return super.getSurcharge().add(SURCHARGE);  
    }  
  
    @Override public List<String> getMetadataLabels() {  
        List<String> labels = super.getMetadataLabels();  
        labels.add("ExtraLegroom");  
        return labels;  
    }  
  
    private String addedBehavior() { return " · 프리미엄 레그룸 (+50,000)"; }  
}
```



-  여기서부터는 **요금을 더하는 옵션**입니다. 앞의 둘과 달리 getSurcharge를 직접 override한
다는 점이 핵심입니다.
-  super.getSurcharge()로 안쪽까지 쌓인 합계를 받은 뒤, 거기에 자기 요금 50,000원을
더합니다. 이 '안쪽 합계 + 내 값' 한 줄이 **요금 누적의 원리 전부**입니다.
-  옵션을 여러 겹 겹치면 이 한 줄이 총마다 실행되며 금액이 차례로 더해집니다. 동시에 문구
와 태그에도 자기 것이 함께 붙습니다.

 LoungeAccessDecorator

라운지 이용 · 요금 +80,000원

LoungeAccessDecorator.java · ConcreteDecorator D

```
public class LoungeAccessDecorator extends AbstractSeatDecorator {  
  
    // ▶ 레그룸과 구조 동일, 금액만 다름  
    private static final BigDecimal SURCHARGE = new BigDecimal("80000");  
  
    public LoungeAccessDecorator(SeatView component) { super(component); }  
  
    @Override public String getDescription() {  
        return super.getDescription() + addedBehavior();  
    }  
  
    // ▶ 안쪽 합계 + 80,000 (똑같은 누적 방식)  
    @Override public BigDecimal getSurcharge() {  
        return super.getSurcharge().add(SURCHARGE);  
    }  
  
    @Override public List<String> getMetadataLabels() {  
        List<String> labels = super.getMetadataLabels();  
        labels.add("Lounge");  
        return labels;  
    }  
  
    private String addedBehavior() { return " · 라운지 이용 (+80,000)"; }  
}
```

요금

안쪽 합계

+

80,000원

=

새 합계

문구

안쪽 문구

+

" · 라운지"

=

...+ 라운지

-  라운지 옵션은 ExtraLegroomDecorator와 구조가 똑같고 금액만 다릅니다(80,000원). 안쪽 합계를 받아 자기 요금을 더하는 방식이 그대로 반복됩니다.
-  한 가지 특별한 규칙이 있습니다. 비즈니스·퍼스트 좌석에는 이 옵션이 자동으로 붙습니다. Client가 좌석 등급을 보고 알아서 한 겹 더 감싸 줍니다.
-  정리하면 A·B(창가·통로)는 라벨만, C·D(레그룸·라운지)는 라벨과 요금을 더합니다. 모두 같은 뼈대를 쓰고 고치는 범위만 다릅니다.

SeatViewBuilder

좌석 정보를 보고 옵션을 한 겹씩 자동으로 쌓는다

SeatViewBuilder.java · Client

```
public final class SeatViewBuilder {

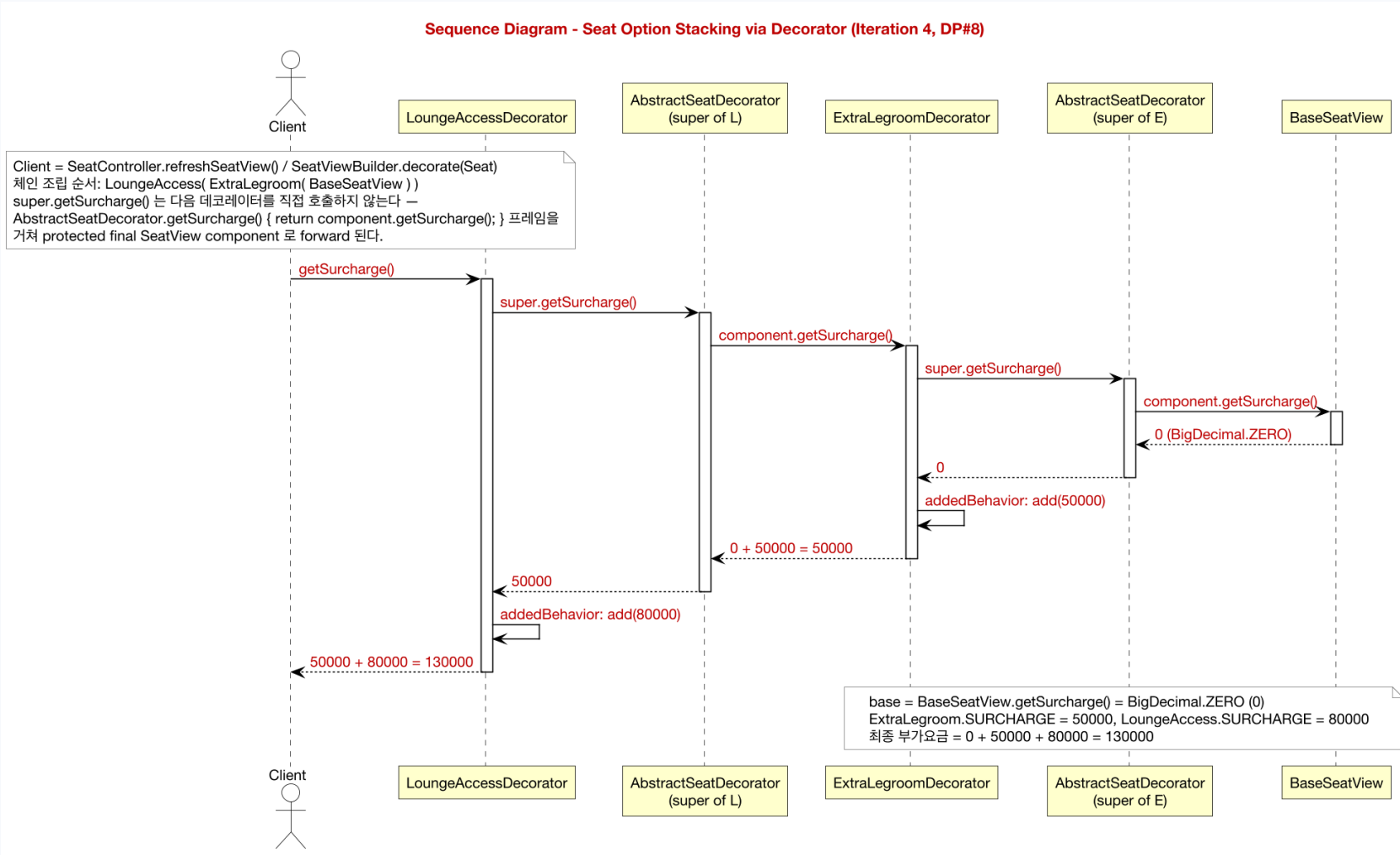
    private SeatViewBuilder() {}

    public static SeatView decorate(Seat seat) {
        SeatView view = new BaseSeatView(seat);           // 기본 좌석에서 출발
        // ▶ ★ view 를 한 겹 더 감쌌 — 이 한 줄이 조립의 핵심
        if (seat.isWindowSeat()) view = new WindowSeatDecorator(view);
        else if (seat.isAisleSeat()) view = new AisleSeatDecorator(view); // 창가/통로 중 하나만
        if (seat.isExtraLegroom()) view = new ExtraLegroomDecorator(view);
        CabinClass cc = seat.getCabinClass();
        // ▶ 상위 등급이면 라운지 자동
        if (cc == CabinClass.BUSINESS || cc == CabinClass.FIRST)
            view = new LoungeAccessDecorator(view);
        return view; // 여러 겹 감싸인 SeatView 하나
    }
}
```



-  핵심은 **view = new XxxDecorator(view)** 한 줄입니다. 지금까지 만든 좌석을 새 옵션으로 감싼 뒤 다시 view에 담아, **한 줄마다 바깥이 한 겹 두꺼워집니다**.
-  어떤 옵션을 씌울지는 좌석 정보가 결정합니다. 창가/통로는 if-else로 묶여 동시에 붙지 않고, 라운지는 등급이 비즈니스·퍼스트일 때 **자동으로** 붙습니다.
-  이게 **Client**인 이유입니다. 조립이 끝나면 화면은 결과인 **SeatView 하나**만 받아 쓰면 되고, 조립의 복잡함은 모두 이 한 곳에 감힙니다.

요금 한 번 계산이 안으로 들어갔다 누적되어 나온다



- ↓ **안으로 들어가는 길.** 가장 바깥 옵션의 getSurcharge()가 호출되면, 자기 요금을 더하기 전에 먼저 자기가 감싼 안쪽(super)에게 요금을 물어봅니다. 그 안쪽도 똑같이 묻고, 한 겹씩 파고 들어가 결국 가장 안쪽 BaseSeatView(0원)에 닿습니다.
- ↑ **밖으로 나오는 길.** BaseSeatView가 0원을 돌려주는 순간부터 호출은 되돌아 나오고, 나오는 길에 각 옵션이 받은 값에 자기 요금을 더합니다. 0원에서 레그룸이 50,000원, 라운지가 80,000원을 보태는 식이라, 이 한 번의 왕복만으로 전체 요금이 정확히 합산됩니다.

🧮 감싸기 체인이 그 좌석의 상태다

비즈니스 · 창가 · 레그룸 좌석 예시

런타임 객체 (안 → 밖)

Lounge(Legroom(Window(BaseSeatView)))
+80,000 +50,000 +0 0 ← 시작

💰 **getSurcharge() — 한 겹씩 누적**

Base
0원

→

+창가
0원

→

+레그룸
50,000

→

+라운지
130,000원

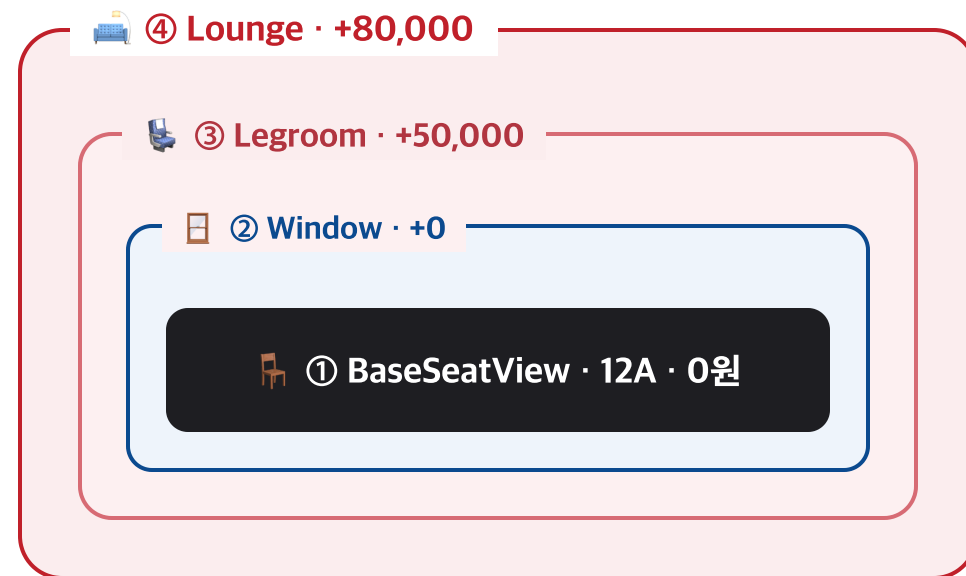
🏷️ **함께 완성되는 문구·태그**

문구 → 12A (BUSINESS) · 창가 · 레그룸 · 라운지
태그 → [Window, ExtraLegroom, Lounge]

이렇게 **겹겹이 감싸인 객체 구조 그 자체가 바로 그 좌석의 현재 상태**입니다. 어떤 옵션을 켜거나 끄는 일은 곧 객체를 한 겹 더 씌우거나 한 겹 벗기는 것과 같고, 그 과정에서 기존 클래스는 단 한 줄도 고치지 않습니다. 덕분에 옵션 조합이 아무리 늘어나도 기존 코드를 건드리지 않는 개방-폐쇄 원칙(OCP)이 자연스럽게 지켜집니다.

🧅 옵션이 기본 좌석을 한 겹씩 두른다

비즈니스 창가+레그룸 좌석을 조립했을 때



바깥쪽에 있을수록 나중에 감싼 옵션입니다. new XxxDecorator(안쪽좌석) 한 줄이 실행될 때마다 한 겹 더 둘러싸기 때문에, 맨 바깥의 라운지가 가장 마지막에 씌워진 것입니다.



이 상태에서 **getSurcharge()**를 한 번 부르면, 호출이 맨 안쪽까지 들어갔다 나오면서 **0원 → +50,000 → +80,000, 최종 130,000원**으로 누적됩니다. 문구와 태그도 같은 방식으로 함께 완성됩니다.



이 모든 게 가능한 이유는, 데코레이터가 **'나도 좌석이면서, 내 안에 또 좌석을 품는다'**는 자기참조 구조이기 때문입니다. 같은 타 입을 끝없이 겹쳐 쌓을 수 있어 조합이 몇 가지든 클래스를 새로 만들지 않습니다.

✈ 객체지향 설계 패턴

대한항공 예약 시스템

Iteration 4 — 디자인 패턴 보고서

Team A · 김정욱 · 이재호 · 김경동

목차

- 팀 기여 · 기능 · Iteration 진행 · 확장 (표)
- 행동 다이어그램 — Use Case / Sequence / State
- 디자인 패턴 DP#1 State ~ DP#9 Decorator (설명 · 비교 · 코드)
- 적용 강도, 현황과 다음 단계

Team Contribution

빨간색 = Iteration 4 작업

김정욱

도메인 모델 및 디자인 패턴 통합

Iter 1-3: 상태 기계, Strategy, Observer.

Iter 4: Composite, Factory Method, Template Method, Decorator; 교과서 구조 리팩토링; Reservation SRP 분리(Extract Class).

이재호

경계(사용자 인터페이스) 계층

Iter 1-3: 콘솔 예약 UI.

Iter 4: FXML 화면 전체 JavaFX UI(app.css, shell 내 비게이션, 화면별 패턴 연결: Decorator 좌석, Factory Method 결제, Observer 버스 연계, State 배지).

김경동

제어 계층, 외부 연동, 품질 보증

Iter 2-3: 결제 처리기, RefundHandler.

Iter 4: Skypass 어댑터, 설정 싱글톤; 컴파일 검증 및 수동 회귀 점검.

 Feature 표

빨간색 = Iteration 4

기능	세부 기능	구현 Iteration
항공편 검색	직항, 경유, 다구간 여정	Iteration 1 Iteration 4 / Refactored / DP#6 (Factory Method)
예약 생애주기	8개 상태 전이 기계	Iteration 1 Iteration 4 / Refactored (state-management)
승객 관리	승객 추가 및 연락처 입력	Iteration 1
결제	카드, ApplePay, KakaoPay, 계좌이체, 마일리지	Iteration 2 Iteration 4 / Refactored / DP#6 (Factory Method)
환불	무환불 / 부분환불 / 전액환불 정책	Iteration 2 / DP#2 (Strategy) Iteration 4 / Refactored
이벤트 알림	연계 버스 티켓, 결제 실패 자동 취소, 항공편 상태 전파	Iteration 3 / DP#3 (Observer) Iteration 4 / Refactored (push to pull)
좌석 선택	창측, 통로측, 추가 레그룸, 라운지 옵션(추가 요금)	Iteration 4 / DP#9 (Decorator)
공항 계층	공항(Leaf)과 다중 공항 도시(Composite)	Iteration 4 / DP#4 (Composite)
e-Ticket 렌더링	평문, HTML, 탑승권 렌더러	Iteration 4 / DP#7 (Template Method)
외부 마일리지	Skypass 회원 검증 및 마일리지 차감	Iteration 4 / DP#8 (Adapter)
전역 설정	단일 공유 설정: 글꼴, 로캘, 통화, 테마	Iteration 4 / DP#5 (Singleton)
사용자 인터페이스	JavaFX 검색/좌석/결제 화면	Iteration 4 / Refactored (JavaFX FXML + CSS)



Iteration Progress 표

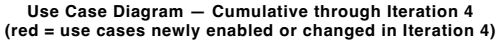
빨간색 = Iteration 4

Iteration	세 부	리팩토링 / 디자인 패턴	적용 위치	요약	기술 페이지
Iteration 1	1-1	DP#1 State + Replace Conditional with Polymorphism (상태 관리)	Reservation, ReservationState	생애주기 전이를 상태 객체로 캡슐화; 상태 조건 분기 제거	Report p.12 · Slide 19
Iteration 2	2-1	DP#2 Strategy Pattern	RefundHandler, RefundPolicy	환불 알고리즘을 교체 가능하게; 새 정책에 열리고 핸들러 수정엔 닫힘	Report p.17 · Slide 21
Iteration 3	3-1	DP#3 Observer Pattern	EventPublisher, listeners	발생원과 반응 분리: 버스 티켓, 자동 취소, 항공편 상태	Report p.19 · Slide 24
Iteration 4	4-0	이전 피드백 반영 — DP#1 State · DP#2 Strategy · DP#3 Observer 리팩토링 및 개선	Reservation, RefundHandler, EventPublisher	이전 Iteration 피드백을 반영해 기존 패턴 구현을 교과서 정합성에 맞게 개선	
Iteration 4	4-1	DP#4 Composite Pattern	AirportLocation, Airport, AirportCity	단일 공항과 다중 공항 도시를 재귀 트리로 동일 취급	Report p.21 · Slide 27
Iteration 4	4-2	DP#5 Singleton Pattern	AppConfig	지연 이중검사 초기화로 전역 설정 인스턴스 하나	Report p.22 · Slide 30
Iteration 4	4-3	Extract Class + DP#6 Factory Method Pattern	PaymentMethodProcessor, ItineraryFactory	각 서브클래스가 자기 산출물 생성; 호출부 타입 분기 제거	Report p.24 · Slide 33
Iteration 4	4-4	DP#7 Template Method Pattern	TicketRenderer	티켓 렌더 골격 고정, 단계 흑만 변경	Report p.26 · Slide 36
Iteration 4	4-5	DP#8 Adapter Pattern	SkypassAdapter	외부 Skypass API를 내부 게이트웨이 인터페이스로 변환	Report p.27 · Slide 39
Iteration 4	4-6	DP#9 Decorator Pattern	SeatView, seat decorators	좌석 옵션(요금/라벨)을 런타임에 누적	Report p.29 · Slide 42
Iteration 4	4-7	Refactored: Extract Class + Replace Push with Pull (교과서 정합)	Reservation, Strategy, Observer, State	ReservationRegistry 추출; pull 모델 Observer; Context 보유 Strategy; State 추상 계층 제거	Report p.32 · Slide 46

Extension Feature 표 (적용 + 향후)

iteration 1 계획 외 확장 — 적용 완료 6건 + 향후 가능성 11건

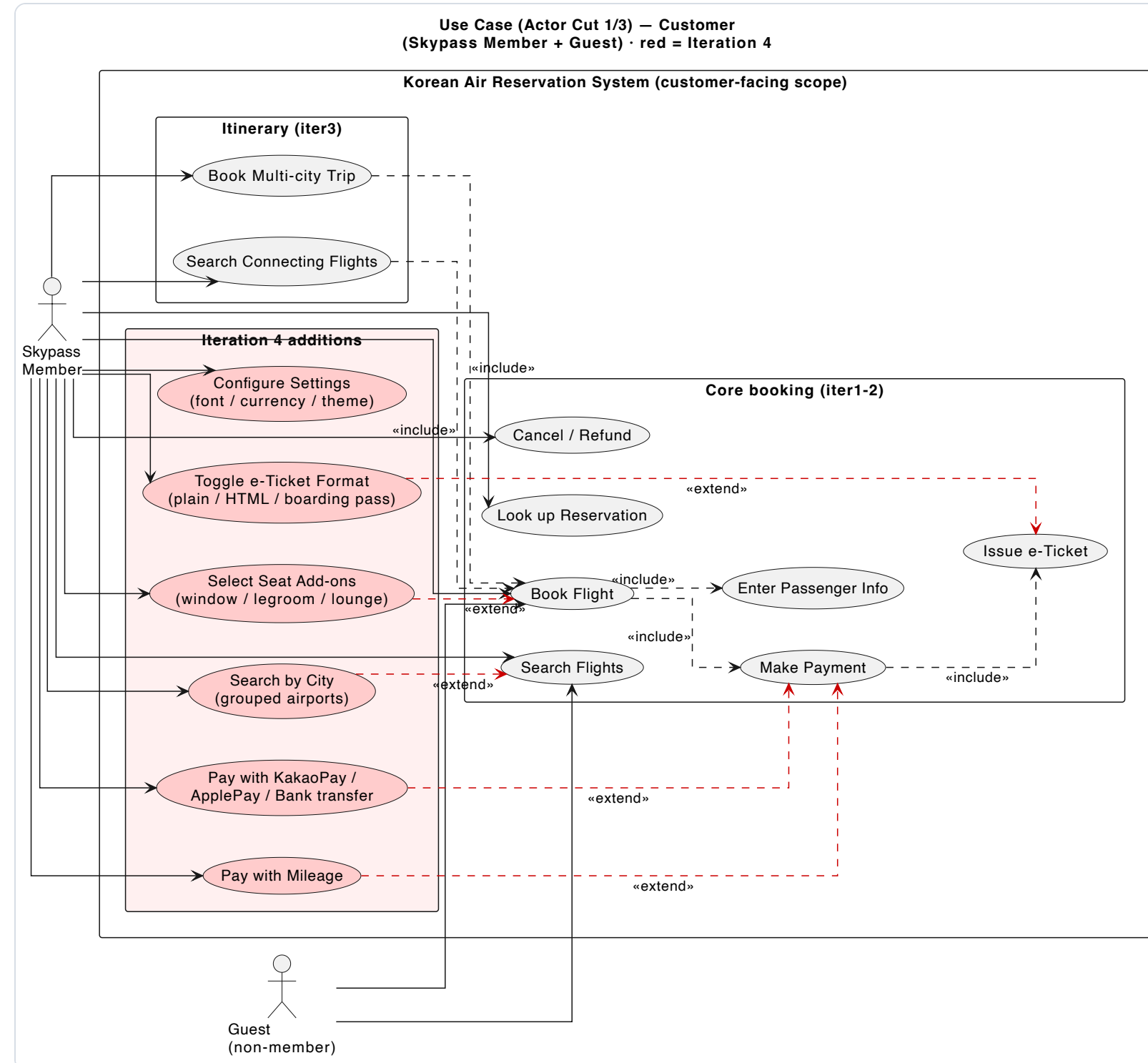
기본 기능	확장 기능	비고
① 이미 적용된 확장 (Refactoring applied) · 빨강 = Iteration 4 추가		
항공편 검색	다중 공항 도시 계층 검색 (도시→복수 공항)	DP#4 Composite 적용 완료 (Iteration 4): 공항=Leaf, 다중공항 도시=Composite 트리 통합 탐색
e-Ticket 발권	다중 포맷 렌더러 (평문 / HTML / 탑승권)	DP#7 Template Method 적용 완료 (Iteration 4): 렌더 골격 고정 + 포맷별 hook 오버라이드
마일리지 결제	외부 Skypass 마일리지 API 연동	DP#8 Adapter 적용 완료 (Iteration 4): 외부 Map 응답을 도메인 boolean으로 변환
좌석 선택	좌석 부가옵션 체인 (창측/통로/레그룸/라운지 추가요금)	DP#9 Decorator 적용 완료 (Iteration 4): 옵션별 요금·설명을 super 위임 후 누적
이벤트 알림	연계 버스 티켓 자동 발권 (6개 대도시 우등고속)	DP#3 Observer 활용, 교수님 지시로 추가 (Iteration 3): 결제 완료 이벤트에 버스티켓 발권 통지
환불	환불 정책 자동 선택 (Resolver 추출)	DP#2 Strategy: RefundPolicyResolver 적용 완료 (Iteration 4), 운임 규칙별 ConcreteStrategy 결정
② 향후 확장 가능성 (Refactoring required) · 기존 패턴 재사용/추가 리팩토링		
예약 생애주기	취소 및 변경 흐름	Refactoring required: 상태 전이 책임 추가 분리 (DP#1 State 기반 상태 관리 리팩토링; 신규 DP 불필요)
알림	이메일 / SMS 알림	DP#3 Observer 재사용; EmailListener, SmsListener 추가 (리팩토링 불필요)
결제	추가 수단 (PayPal, 암호화폐)	DP#6 Factory Method 재사용; ConcreteCreator/ConcreteProduct 추가
좌석	좌석 추천 엔진	Reuse DP#2 (랭킹용 Strategy) + 서비스 클래스
할인	회원 등급 할인	Reuse DP#2 (가격 Strategy)
관리	예약 통계 대시보드	새 read-model 서비스; 실시간 집계에 DP#3 Observer 가능
검색	예약 이력 검색	Refactoring required: Repository / Specification 추출; 신규 DP 불필요
결제 연동	외부 결제대행(PG) API	DP#8 Adapter 재사용; PG마다 Adapter 하나
국제화	다국어 UI	Refactoring required: 리소스 번들 추출; 로캘에 DP#5 설정 싱글턴 재사용
관측성	로깅 / 감사 추적	Expected Decorator(DP#9) 또는 Proxy 후보
신뢰성	강화된 예외 처리	Refactoring required: 횡단 관심사 오류 처리; 신규 DP 불필요



액터는 **Customer**(Skypass Member / Guest), Administrator, 외부 legacy 시스템(Payment Gateway · Skypass System ·

👋 Use Case — Customer (액터 컷 1/3)

Skypass Member · Guest 대면 유스케이스



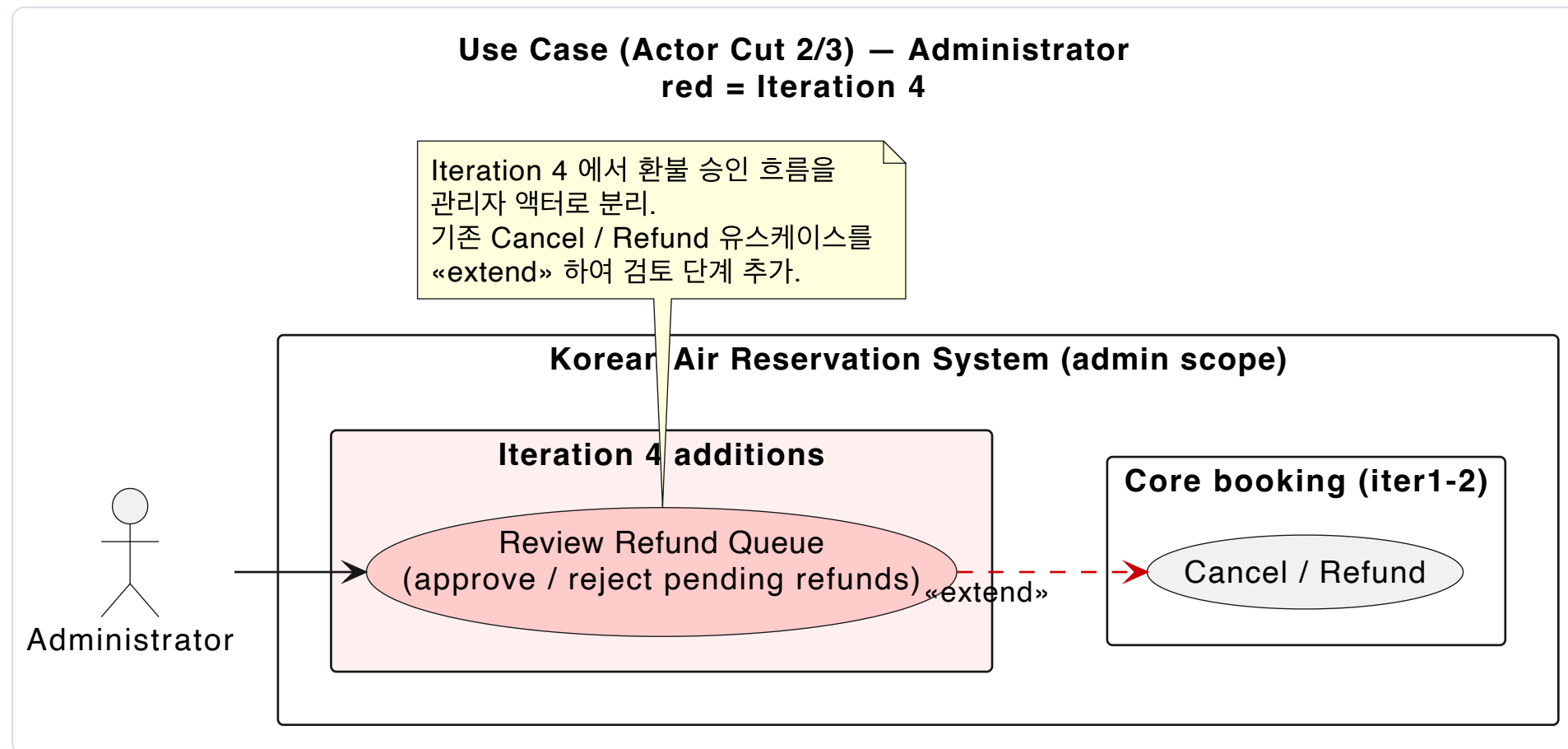
Member는 검색·예약·조회·취소/환불 전 범위를,
Guest는 검색·예약까지 사용합니다.

예약은 **<<include>>**로 승객 정보·결제를, 결제는 발권을 항상 포함합니다.

빨간색 Iteration 4 기능은 모두 **<<extend>>** — 간편/마일리지 결제는 결제를, 좌석 옵션은 예약을, 도시 검색은 검색을 *선택적으로* 확장합니다.

Use Case — Administrator (액터 컷 2/3)

관리자 대면 유스케이스

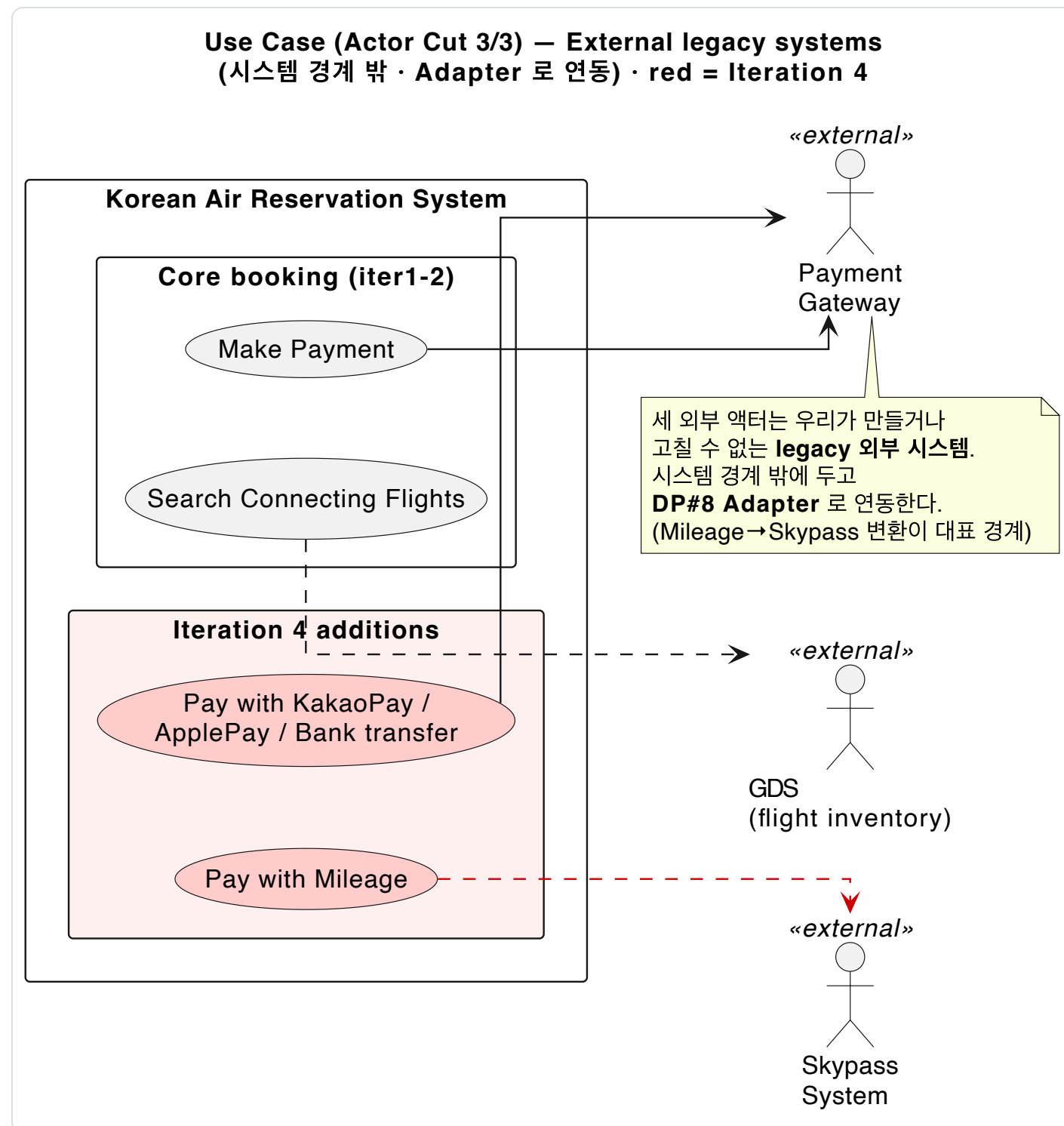


Iteration 4에서 **환불 승인 흐름**을 **Administrator** 액터로 분리했습니다.

Review Refund Queue는 기존 **Cancel / Refund** 유스케이스를 **<<extend>>**해서, 고객이 신청한 환불을 관리자가 승인·반려하는 검토 단계를 추가합니다.

🔌 Use Case — External 경계 (액터 컷 3/3)

외부 legacy 시스템 · Adapter 연동 지점



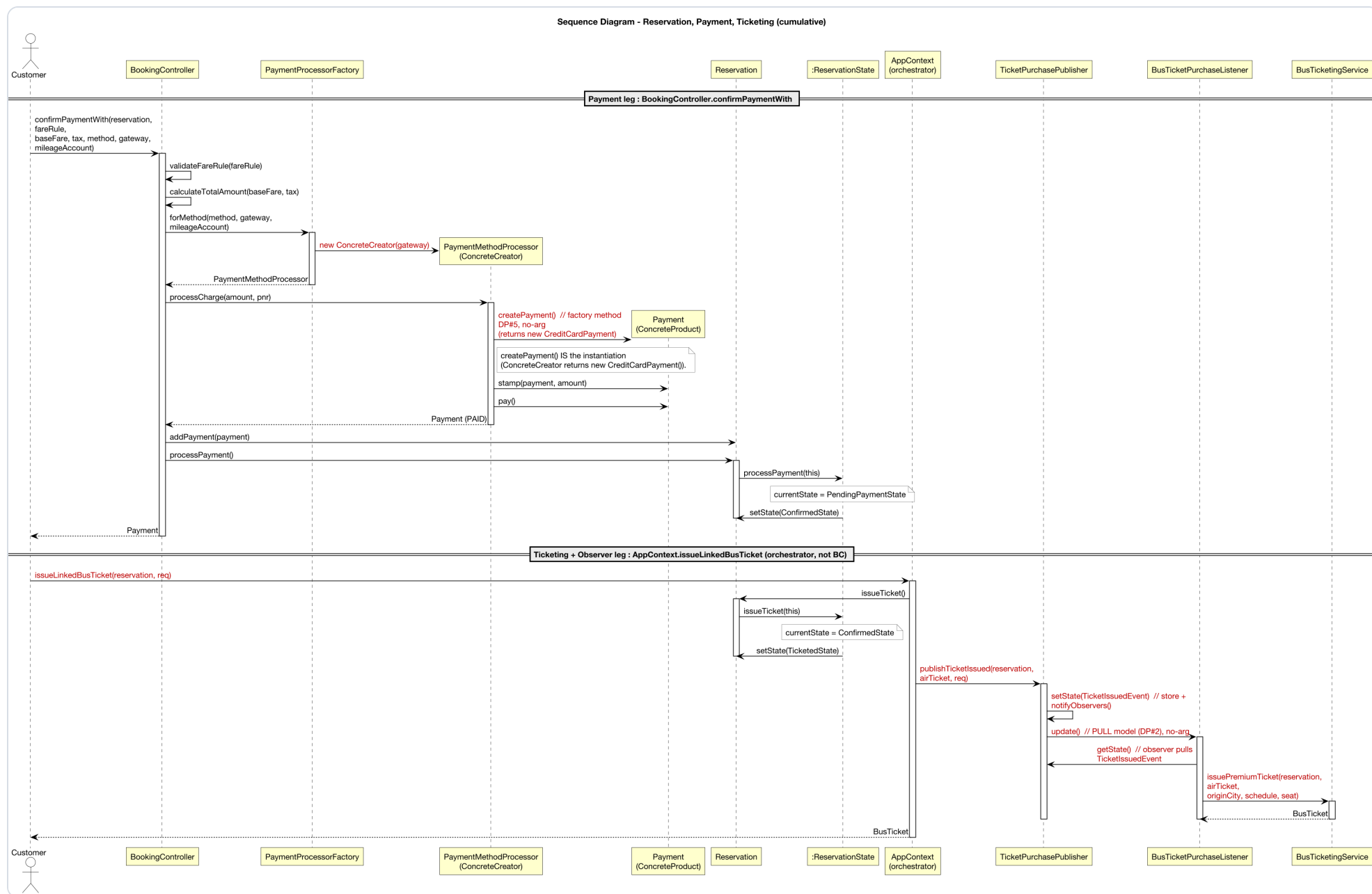
Payment Gateway · Skypass System · GDS는 우리가 만들거나 고칠 수 없는 **legacy 외부 시스템**입니다. 시스템 경계 *밖*에 둡니다.

경계를 넘는 지점: **마일리지 결제→Skypass**, 간편 결제 →Payment Gateway, 연결편 검색→GDS.

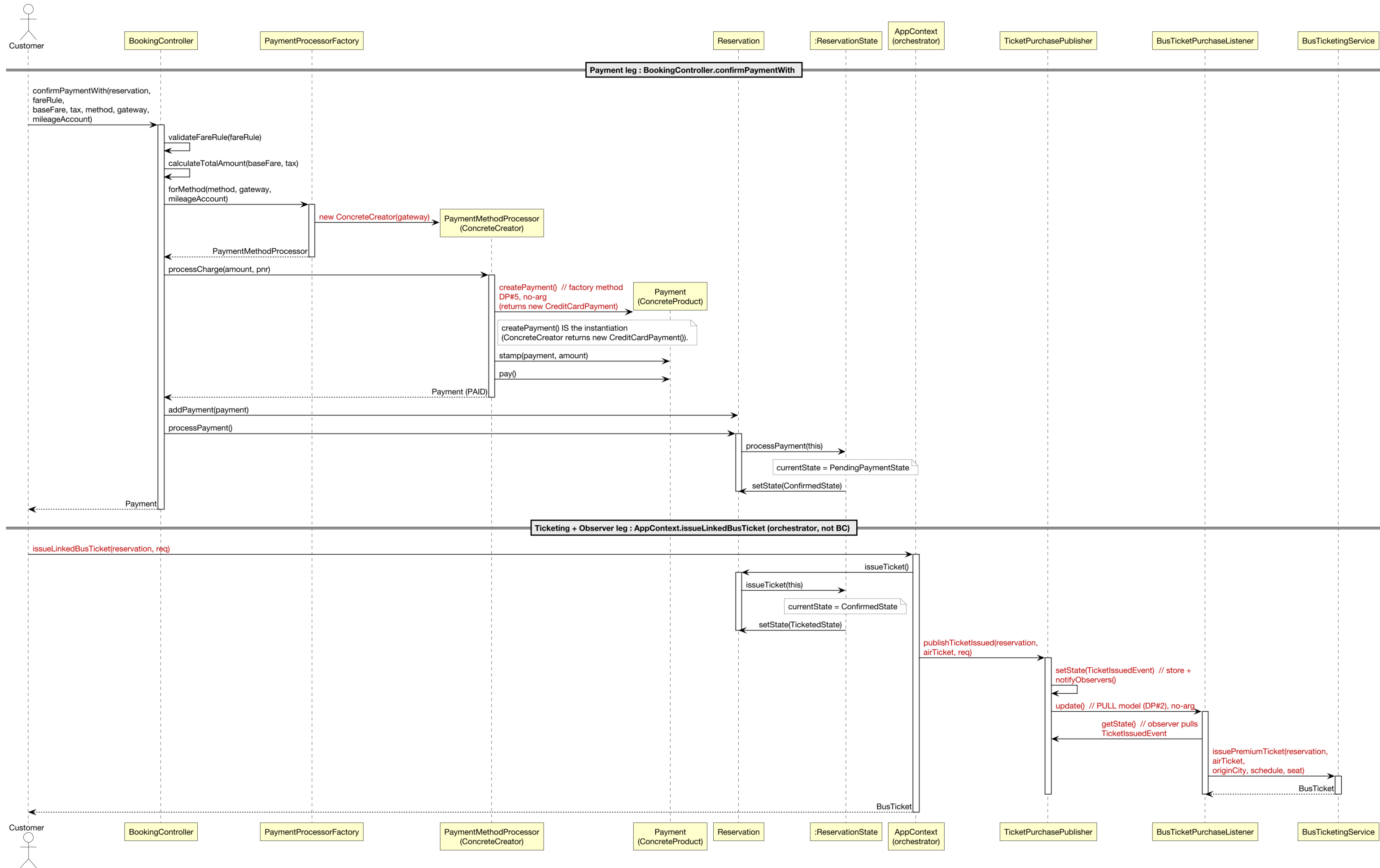
특히 **Mileage→Skypass** 변환이 뒤에 나올 **DP#8 Adapter**가 책임지는 대표 경계입니다.

Sequence — 예약 / 결제 / 발권

주요 시나리오, 시간 순



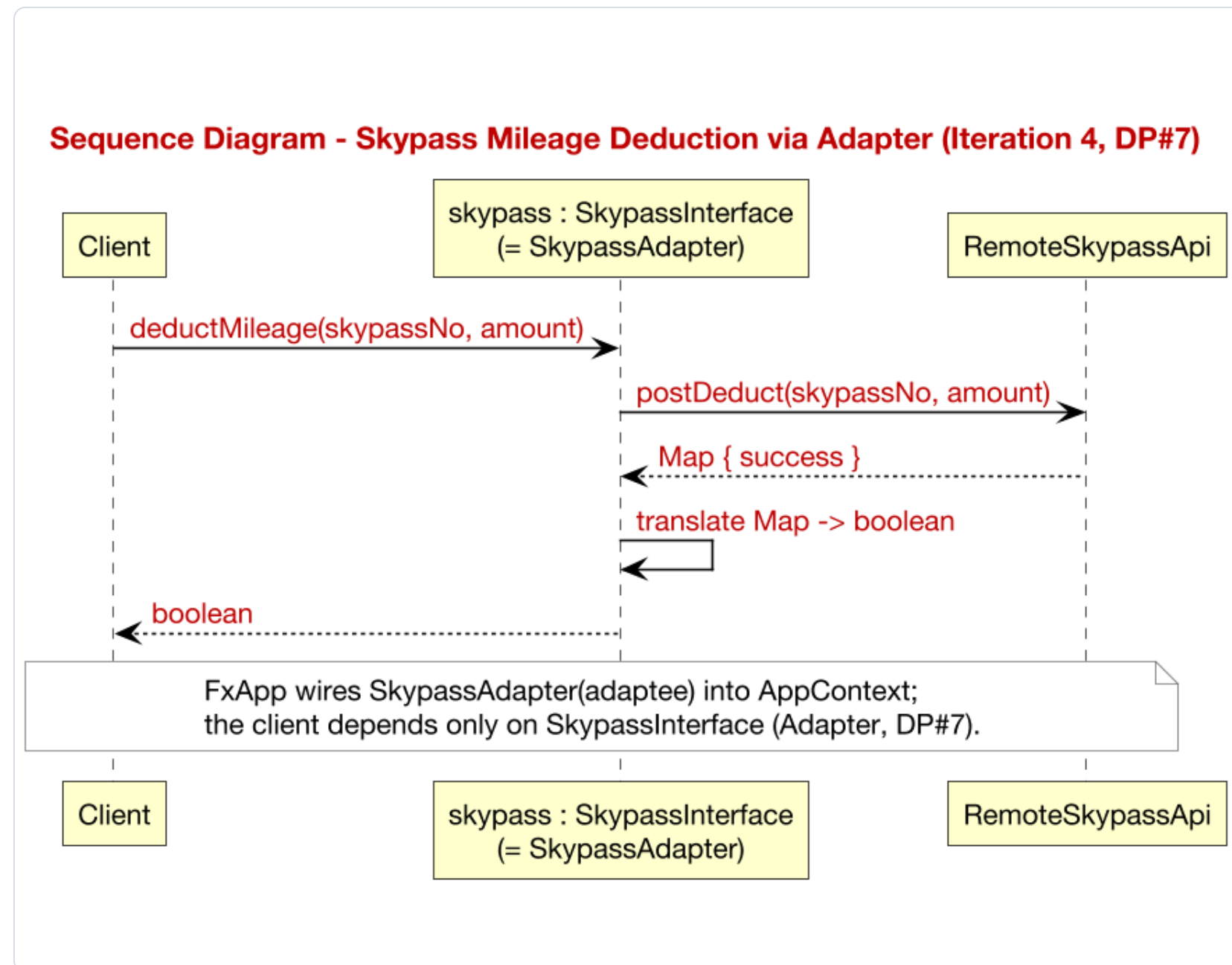
결제 객체는 **Factory Method**(무인자 createPayment)가 생성, 발권 통지는 **Observer** pull(무인자 notifyObservers / update 후 getState).



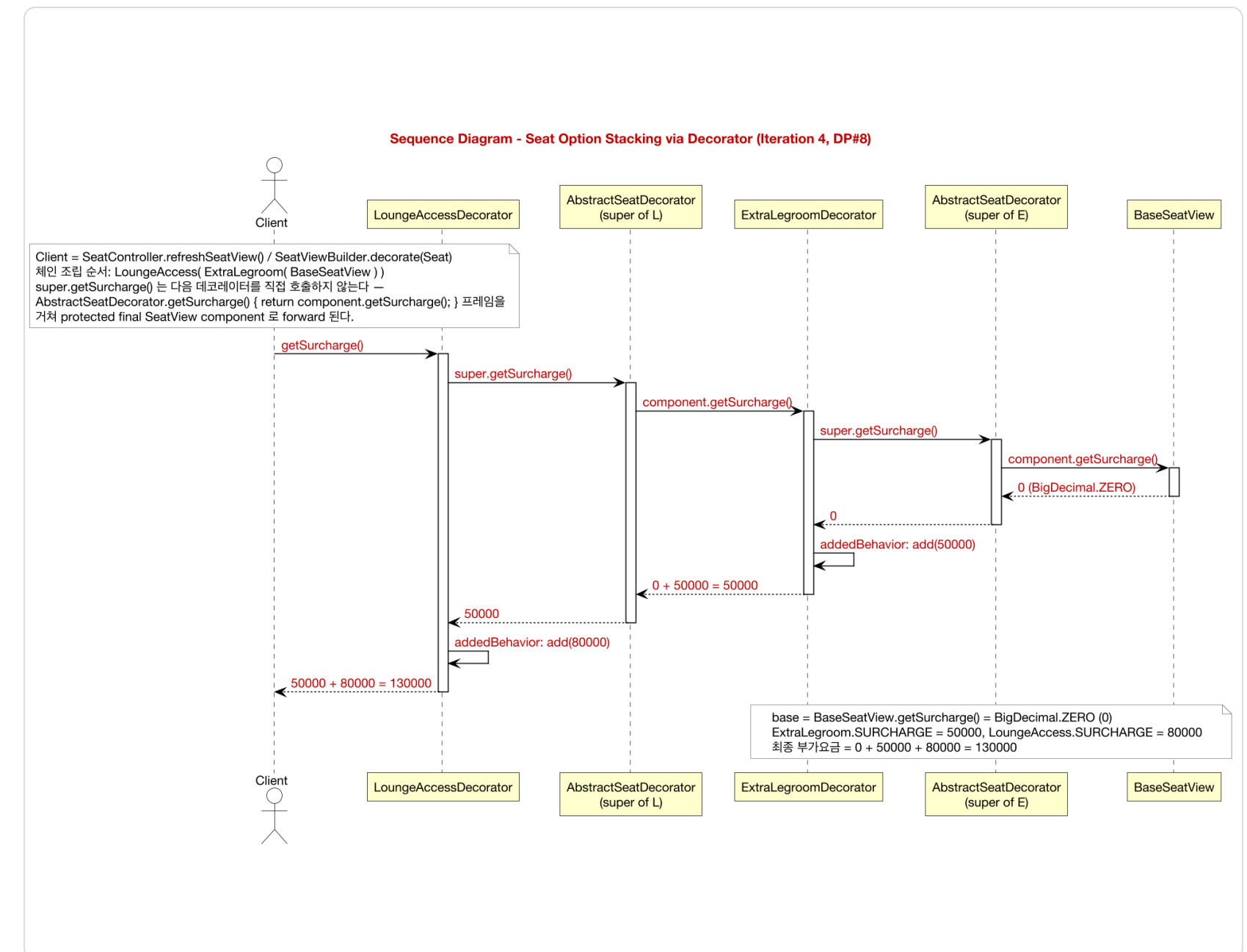
Sequence — Adapter & Decorator

Iteration 4 두 흐름

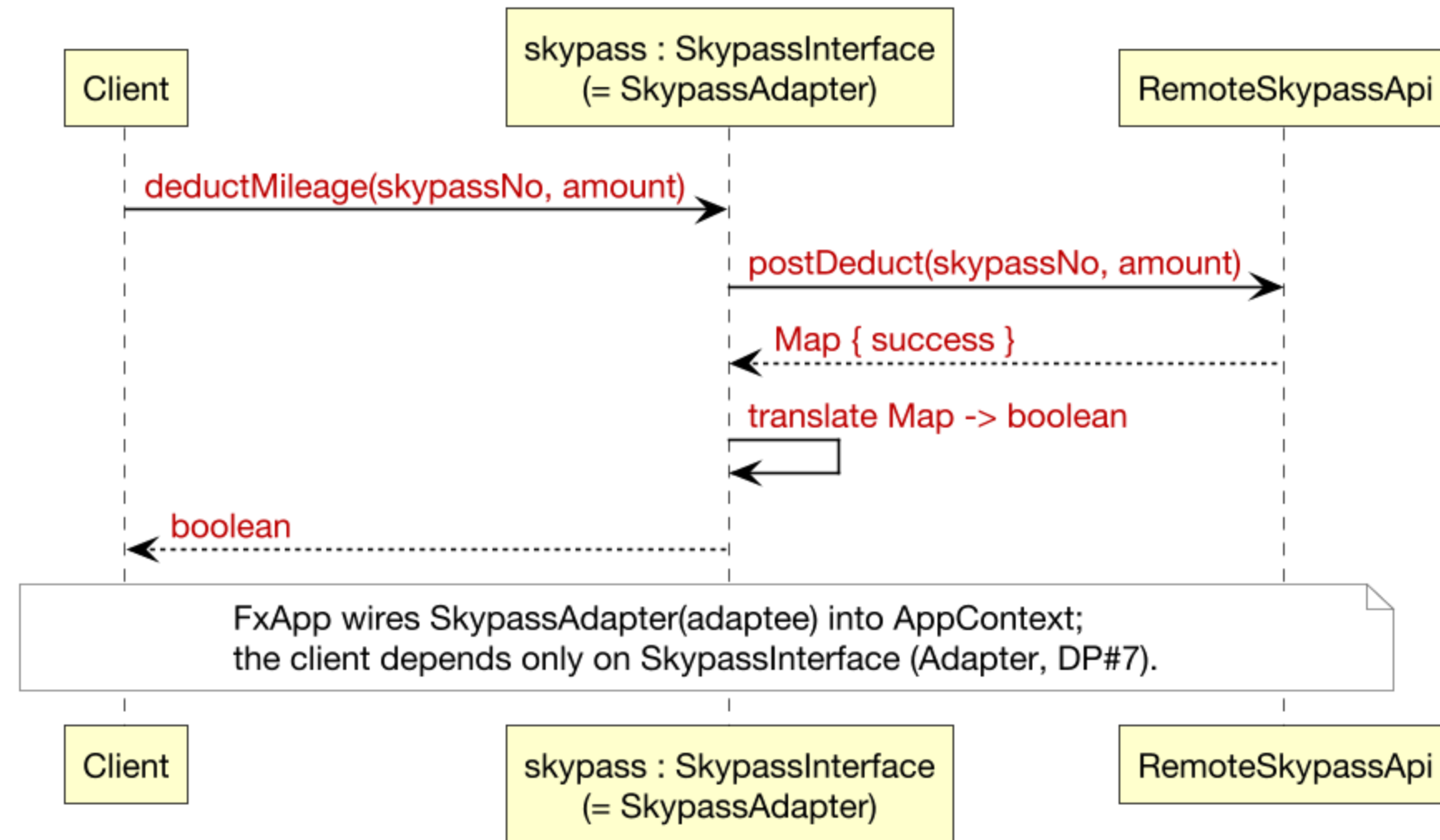
마일리지 Adapter

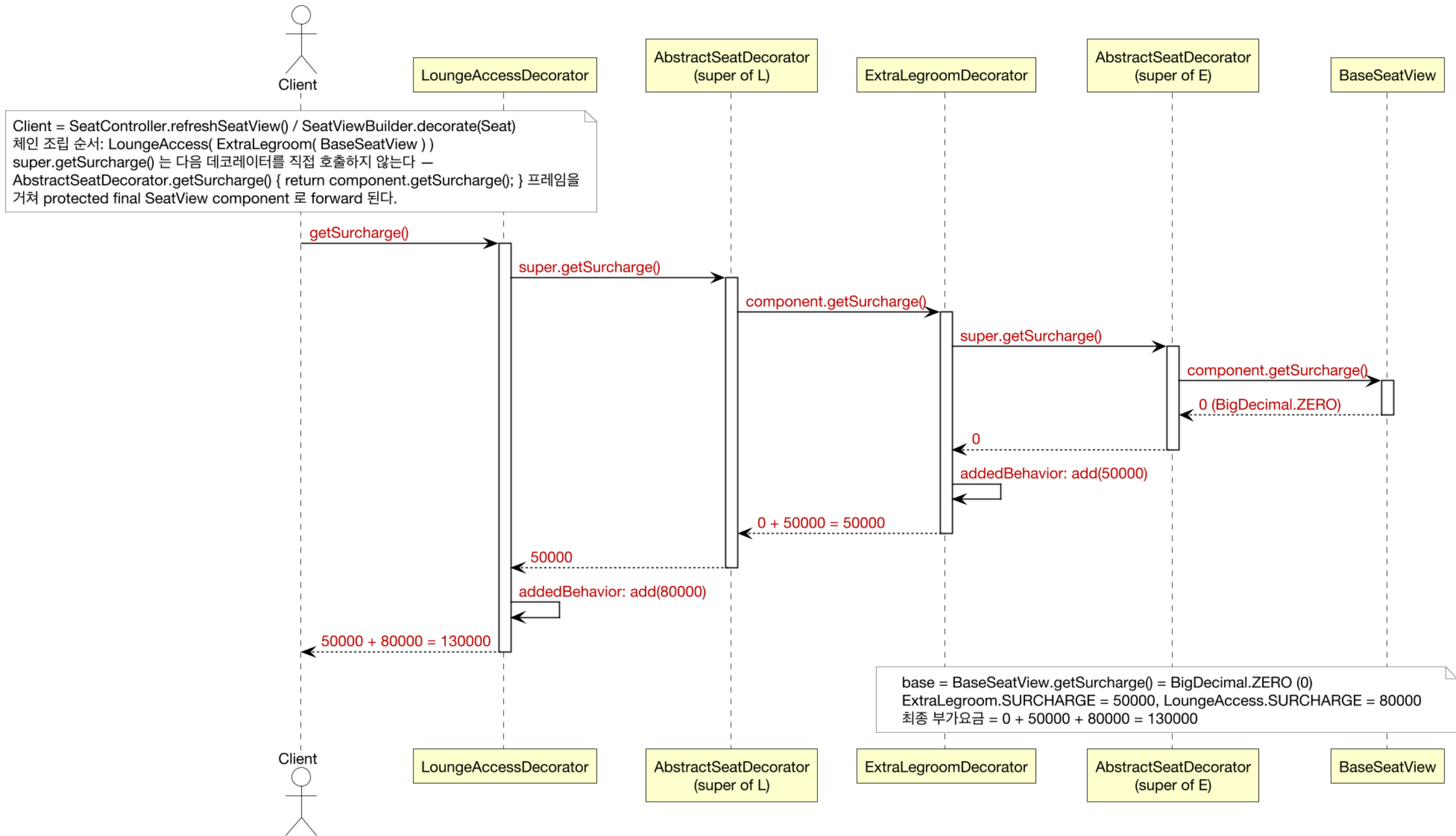


좌석 Decorator



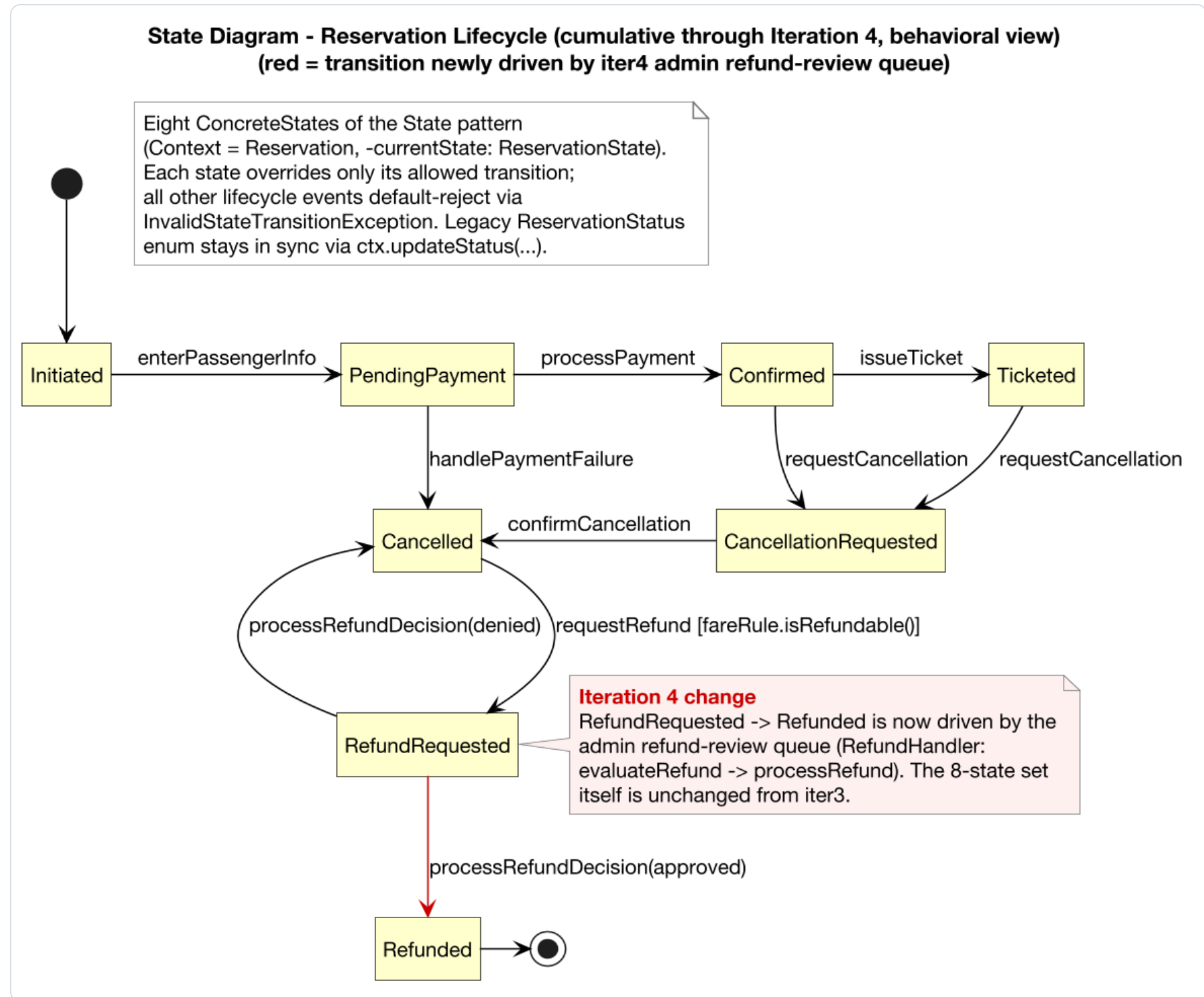
Adapter: SkypassAdapter가 adaptee.postDeduct Map을 boolean으로 변환 / Decorator: 각 옵션이 super 위임 후 요금 누적.



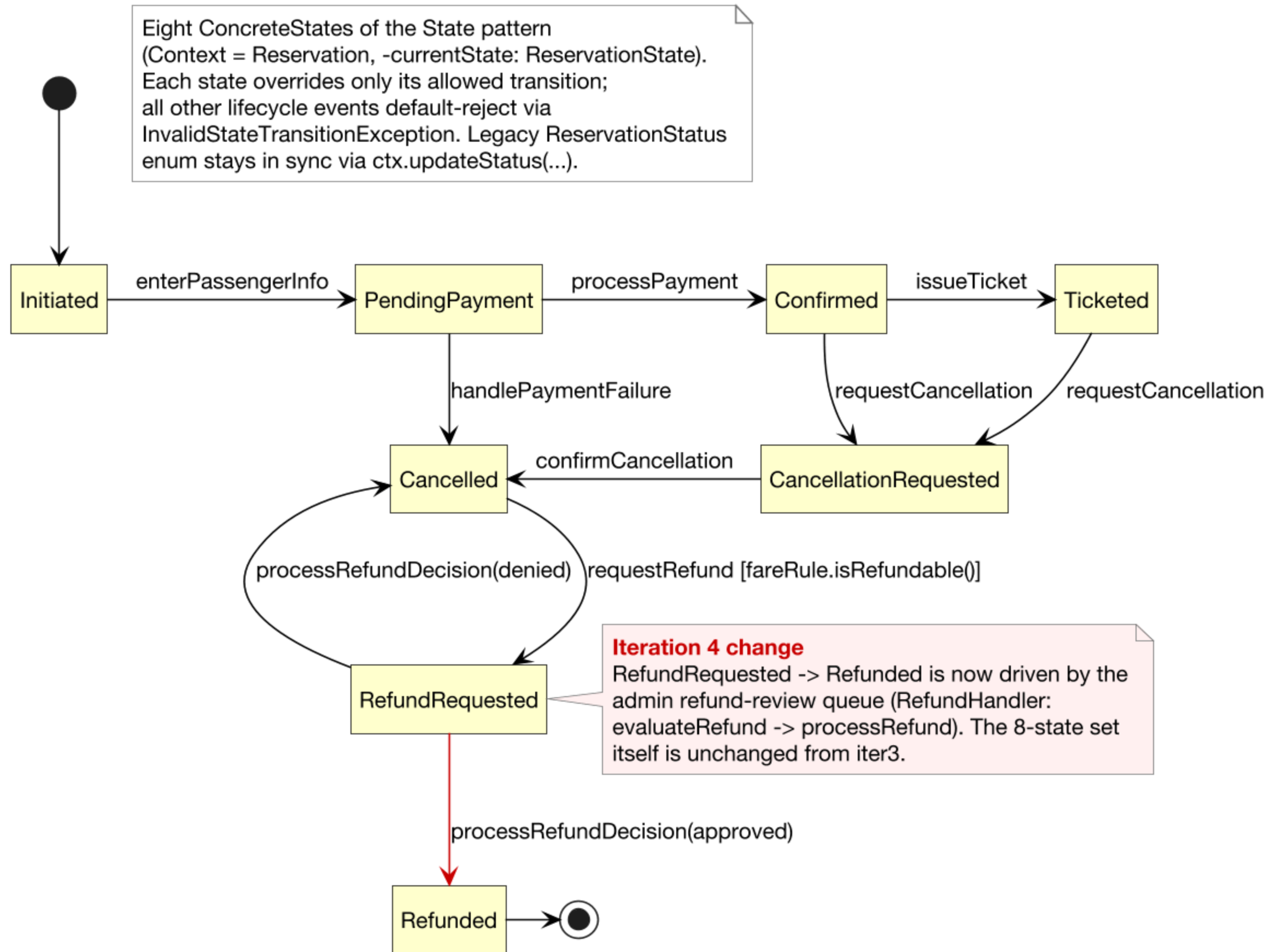


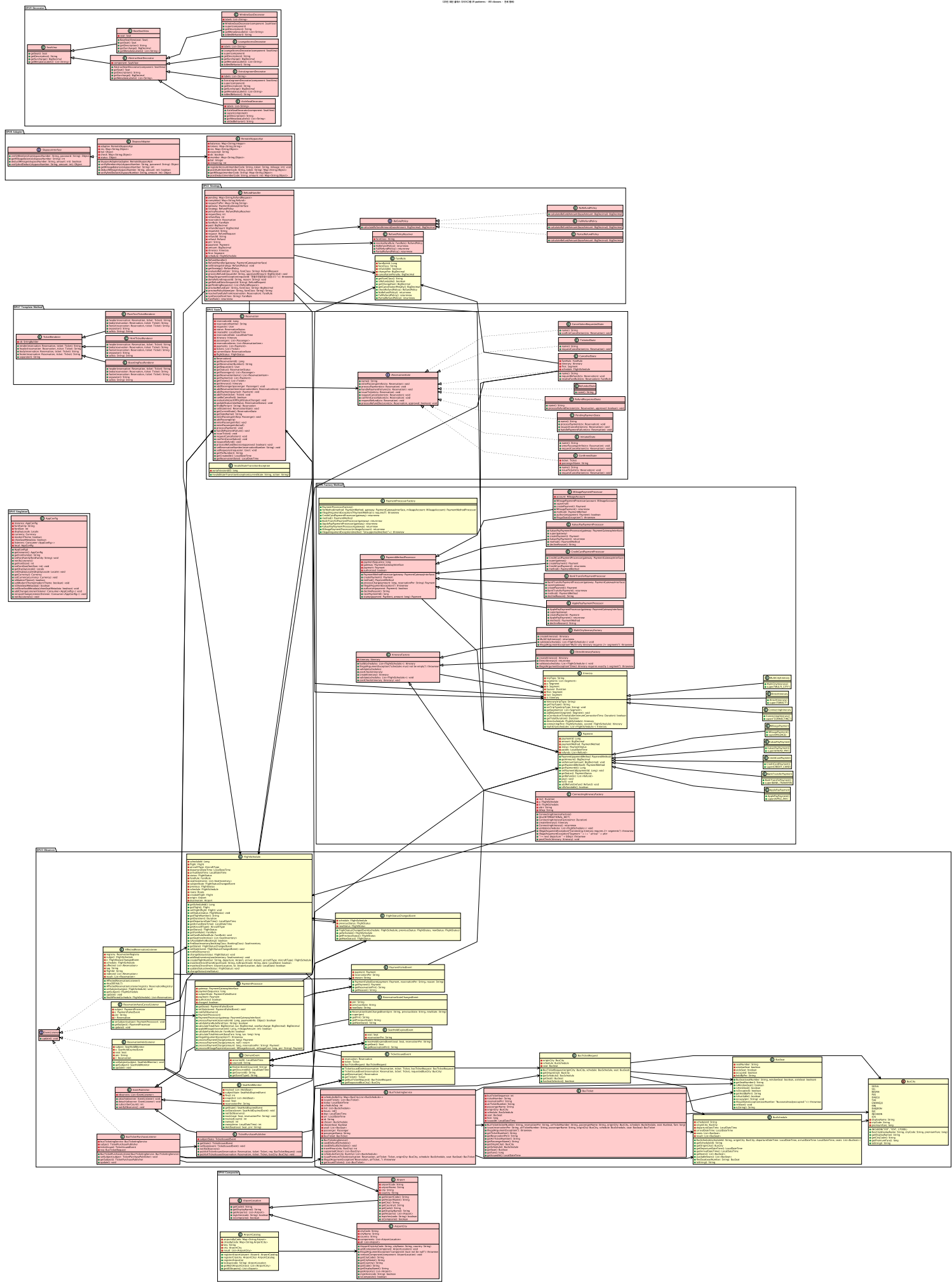
예약 상태 머신

8개 상태, 행동 관점



Initiated~Refunded 8개 상태. 각 전이는 상태 객체가 처리, 허용 안 되는 전이는 **InvalidStateTransitionException**.



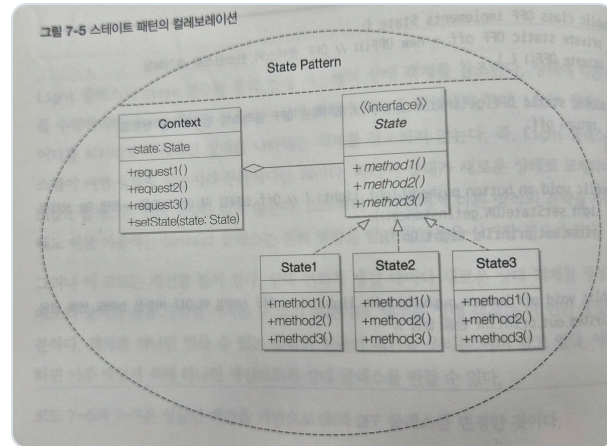




DP#1 State 구현

Context = Reservation · DP#1

교과서 (그림 7-5)



Context = Reservation, **State** = ReservationState 인터페이스 → 8개 ConcreteState (중간 추상 계층 없음).

- Replace Conditional with Polymorphism 리팩토링

구현

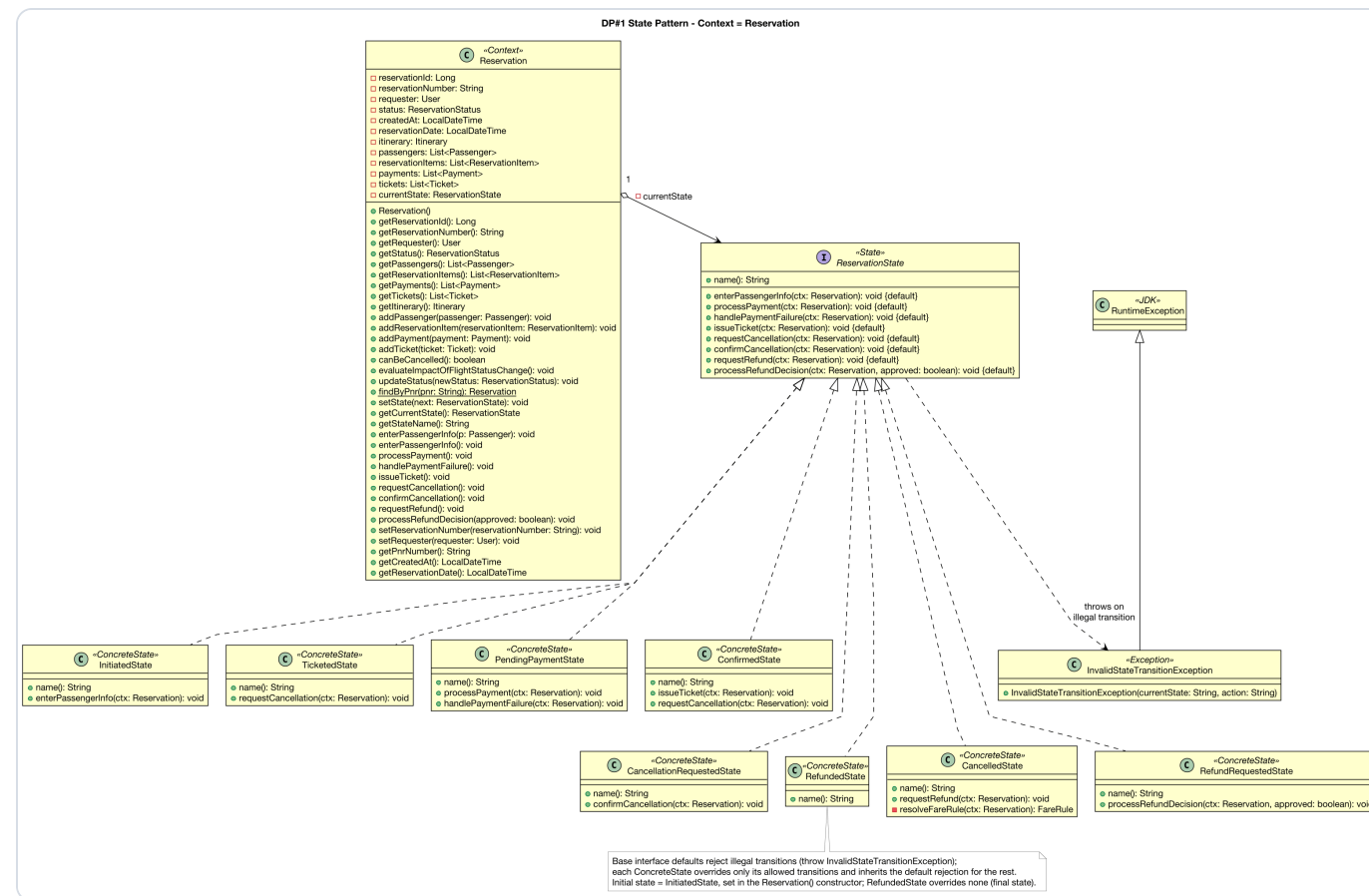
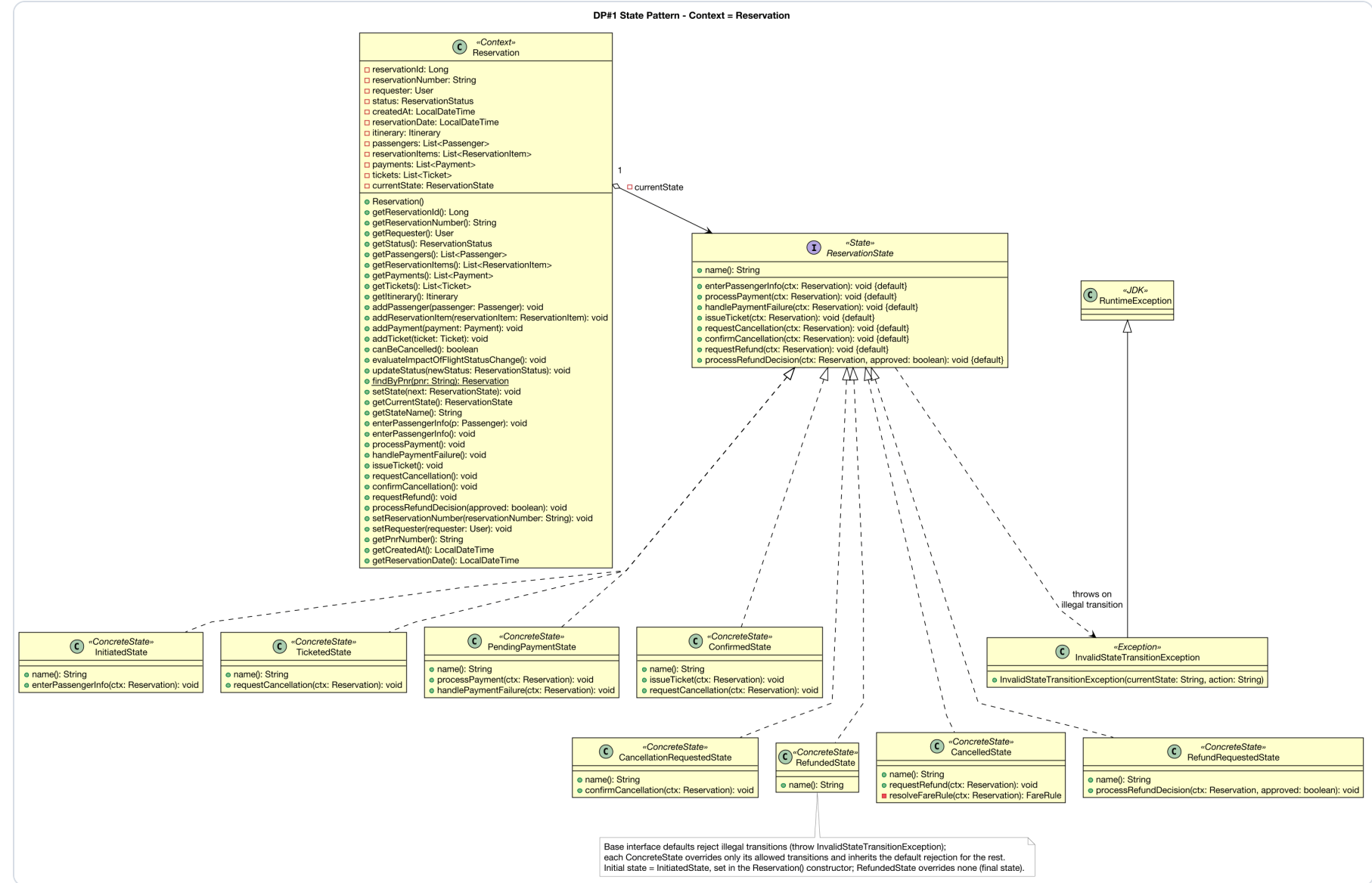
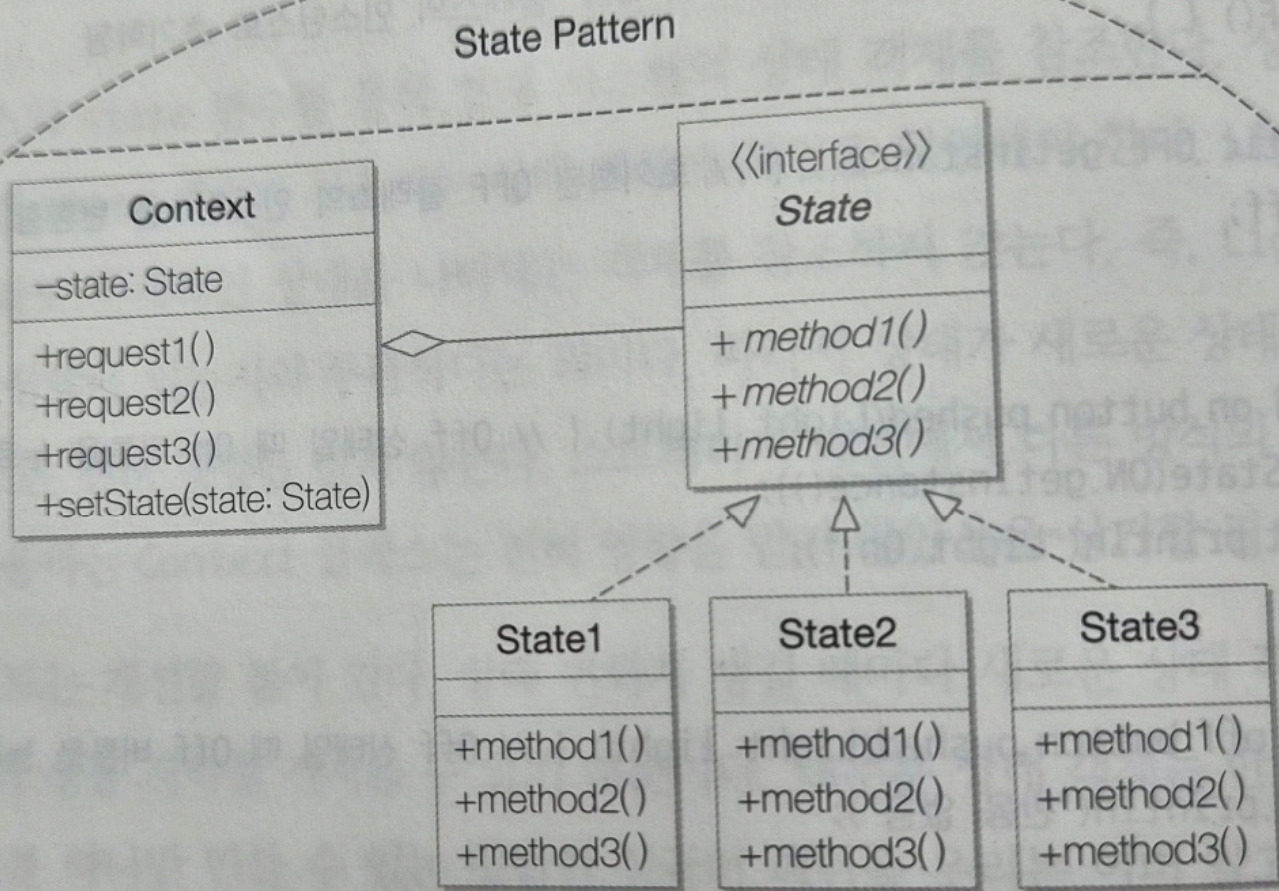


그림 7-5 스테이트 패턴의 컬레보레이션



An abacus with five vertical rods. From left to right, the rods show 1 bead in the top row, 2 beads in the top row, 3 beads in the middle row, 4 beads in the middle row, and 5 beads in the bottom row. This represents the number 12345.

© 2010 Pearson Education, Inc. or its affiliate(s). All rights reserved.

그림 5-6 스트래티지 패턴 클래스 다이어그램

```

classDiagram
    class Context {
        +ContextMethod()
    }
    class StrategyInterface {
        <<interface>>
        +StrategyMethod()
    }
    class ConcreteStrategy1 {
        +StrategyMethod()
    }
    class ConcreteStrategy2 {
        +StrategyMethod()
    }
    class ConcreteStrategy3 {
        +StrategyMethod()
    }
    Context o--> StrategyInterface : -strategy
    StrategyInterface <|-- ConcreteStrategy1
    StrategyInterface <|-- ConcreteStrategy2
    StrategyInterface <|-- ConcreteStrategy3
  
```

The diagram illustrates the Strategy Pattern. It features a **Context** class with a `+ContextMethod()` operation. The **Context** class holds a reference to a **Strategy** interface, indicated by a dashed arrow labeled `-strategy`. The **Strategy** interface defines the `+strategyMethod()` operation. Three concrete classes, **ConcreteStrategy1**, **ConcreteStrategy2**, and **ConcreteStrategy3**, implement the **Strategy** interface, as shown by solid arrows pointing from each concrete class to the interface. Each concrete strategy class also implements the `+strategyMethod()` operation.

- RefundPolicyResolver가 정책 선택, RefundHandler는 setStrategy + delegate만
- 핸들러 수정 없이 새 정책 추가 (OCP)

DP#2 Strategy – 환불 정책 알고리즘 교체

```

classDiagram
    class RefundHandler {
        <<Context>>
        +pending: Map<String, RefundRequest>
        +completed: Map<String, Refund>
        +requestToPnr: Map<String, String>
        +gateway: PaymentGatewayInterface
        +strategy: RefundPolicy
        +requestSeq: int
        +refundSeq: int
        +REFUND_YYMM: DateTimeFormatter
        +setStrategy(): void
        +getStrategy(): RefundPolicy
        +evaluateRefund(): RefundRequest
        +processRefund(): void
        +denyRefund(): void
        +getRefundDetail(): RefundRequest
        +getPendingRequests(): List<RefundRequest>
        +previewRefund(): BigDecimal
        +previewPolicyName(): String
        +resolvePolicy(): RefundPolicy
        +resolveFareRuleFrom(): FareRule
        +synthesize(): FareRule
    }
    class RefundPolicy {
        <<Strategy>>
        +calculateRefundAmount(): BigDecimal
        +getRefundType(): String
    }
    class RefundPolicyResolver {
        <<Selector>>
        +resolve(): RefundPolicy
    }
    class FullRefundPolicy {
        +calculateRefundAmount(): BigDecimal
        +getRefundType(): String
    }
    class PartialRefundPolicy {
        +HALF: BigDecimal
        +calculateRefundAmount(): BigDecimal
        +getRefundType(): String
    }
    class NoRefundPolicy {
        +calculateRefundAmount(): BigDecimal
        +getRefundType(): String
    }
    class FareRule {
        +fareRuleId: Long
        +fareClass: String
        +refundable: boolean
        +changeFee: BigDecimal
        +cancellationPenalty: BigDecimal
        +getFareClass(): String
        +isRefundable(): boolean
        +getChangeFee(): BigDecimal
        +getCancellationPenalty(): BigDecimal
        +checkRefundPolicy(): RefundPolicy
    }

    RefundHandler o--> RefundPolicy : strategy
    RefundHandler ..> RefundPolicyResolver : 정책 선택 위임
    RefundPolicyResolver ..> RefundPolicy : FareRule 기반 선택
    RefundPolicy <|.. FullRefundPolicy
    RefundPolicy <|.. PartialRefundPolicy
    RefundPolicy <|.. NoRefundPolicy
    FareRule ..> RefundPolicy : 등급이 정책 결정
    
```

RefundHandler (Context)

- pending: Map<String, RefundRequest>
- completed: Map<String, Refund>
- requestToPnr: Map<String, String>
- gateway: PaymentGatewayInterface
- strategy: RefundPolicy
- requestSeq: int
- refundSeq: int
- REFUND_YYMM: DateTimeFormatter
- setStrategy(): void
- getStrategy(): RefundPolicy
- evaluateRefund(): RefundRequest
- processRefund(): void
- denyRefund(): void
- getRefundDetail(): RefundRequest
- getPendingRequests(): List<RefundRequest>
- previewRefund(): BigDecimal
- previewPolicyName(): String
- resolvePolicy(): RefundPolicy
- resolveFareRuleFrom(): FareRule
- synthesize(): FareRule

RefundPolicy (Strategy)

- calculateRefundAmount(): BigDecimal
- getRefundType(): String

RefundPolicyResolver (Selector)

- resolve(): RefundPolicy

FullRefundPolicy

- calculateRefundAmount(): BigDecimal
- getRefundType(): String

PartialRefundPolicy

- HALF: BigDecimal
- calculateRefundAmount(): BigDecimal
- getRefundType(): String

NoRefundPolicy

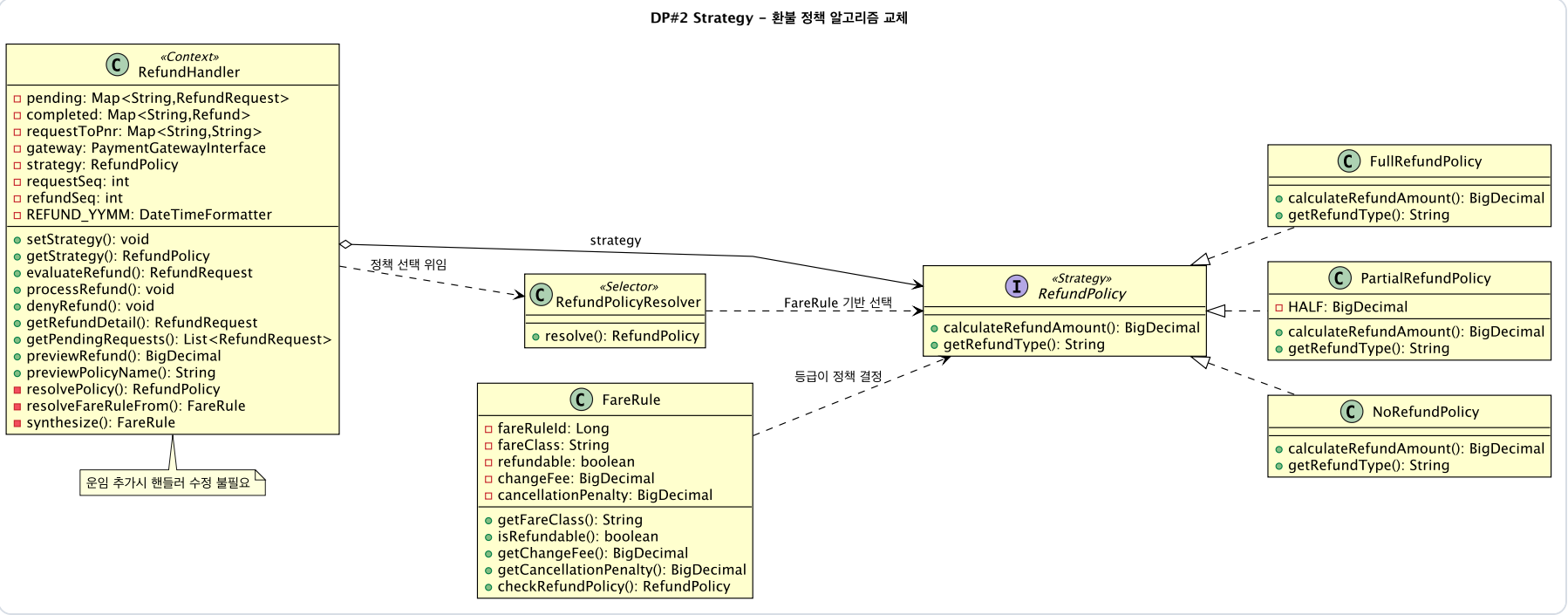
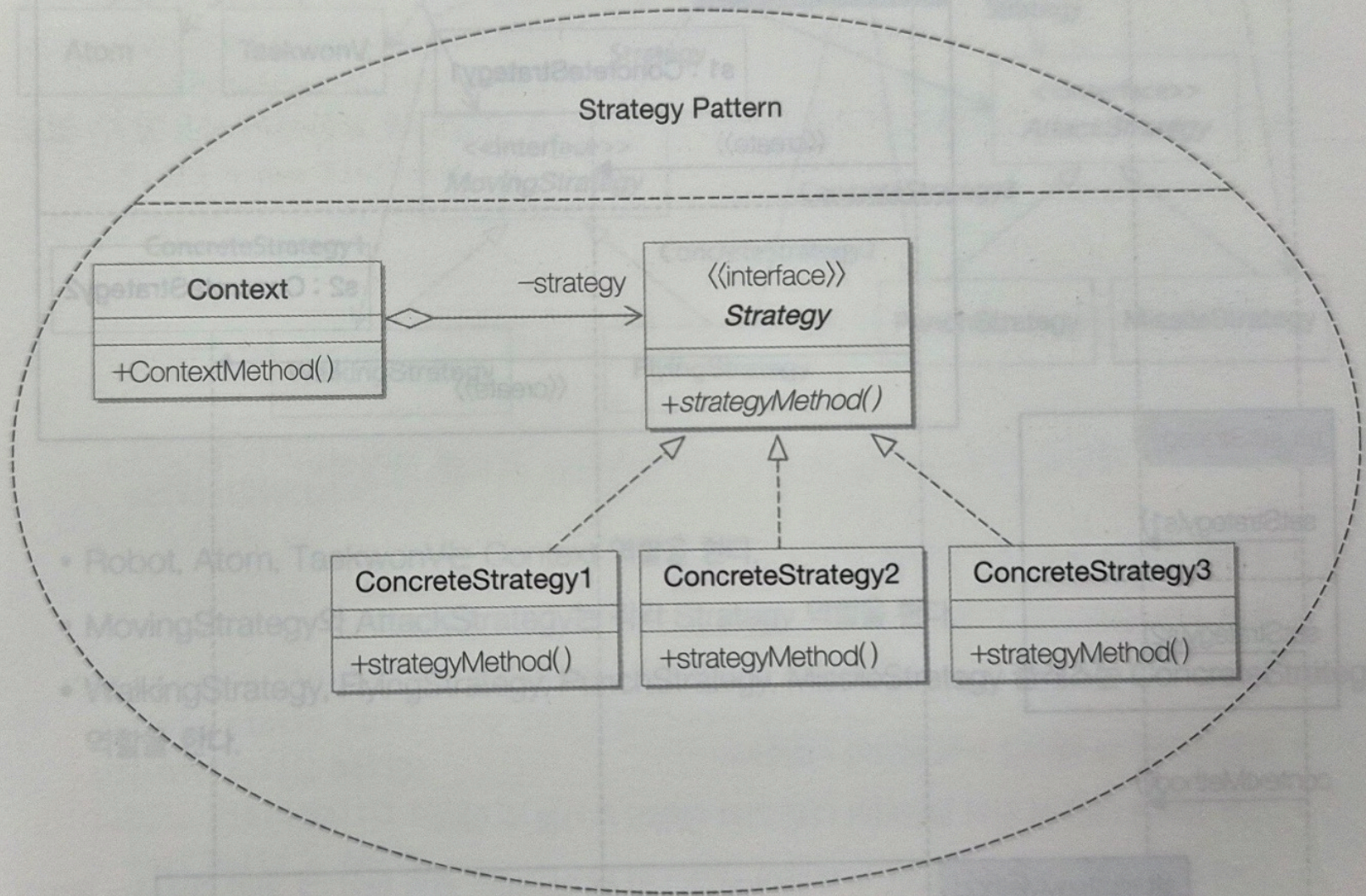
- calculateRefundAmount(): BigDecimal
- getRefundType(): String

FareRule

- fareRuleId: Long
- fareClass: String
- refundable: boolean
- changeFee: BigDecimal
- cancellationPenalty: BigDecimal
- getFareClass(): String
- isRefundable(): boolean
- getChangeFee(): BigDecimal
- getCancellationPenalty(): BigDecimal
- checkRefundPolicy(): RefundPolicy

운임 추가시 핸들러 수정 필요요

그림 5-6 스트래티지 패턴 컬레보레이션





DP#2 Strategy — 코드 (관련 클래스 전부)

Context=RefundHandler, Strategy=RefundPolicy (No/Partial/Full).

RefundPolicy.java

Strategy · 전략 인터페이스

```
// 모든 환불 정책이 구현하는 공통 알고리즘 타입.
// Context(RefundHandler)는 이 타입에만 의존하고 구체 정책은 모른다.
public interface RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal baseAmount);
}
```

No / Partial / FullRefundPolicy.java

ConcreteStrategy · 교체 가능한 3개 알고리즘

```
// 환불 불가 — 항상 0원
public class NoRefundPolicy implements RefundPolicy {
    public BigDecimal calculateRefundAmount(BigDecimal base) { return BigDecimal.ZERO; }
}

// 부분 환불 — 결제액의 50%
public class PartialRefundPolicy implements RefundPolicy {
    public BigDecimal calculateRefundAmount(BigDecimal base) {
        return base.multiply(HALF).setScale(0, RoundingMode.HALF_UP); // ×0.5
    }
}

// 전액 환불 — 100%
public class FullRefundPolicy implements RefundPolicy {
    public BigDecimal calculateRefundAmount(BigDecimal base) { return base; }
}
```

RefundHandler.java

Context · 전략 보유, 위임만 수행

```
// 현재 전략을 필드로 보유(-strategy). 디폴트 NoRefundPolicy.
private RefundPolicy strategy = new NoRefundPolicy();
private final RefundPolicyResolver policyResolver = new RefundPolicyResolver();

public void setStrategy(RefundPolicy strategy) { // 런타임 교체
    this.strategy = strategy;
}

// 선택은 Resolver에, 계산은 보유 전략에 위임 — Context엔 정책 분기 없음
setStrategy(policyResolver.resolve(fareRule));
BigDecimal refund = this.strategy.calculateRefundAmount(paid);
```

RefundPolicyResolver.java

Selector · 정책 선택 책임 분리

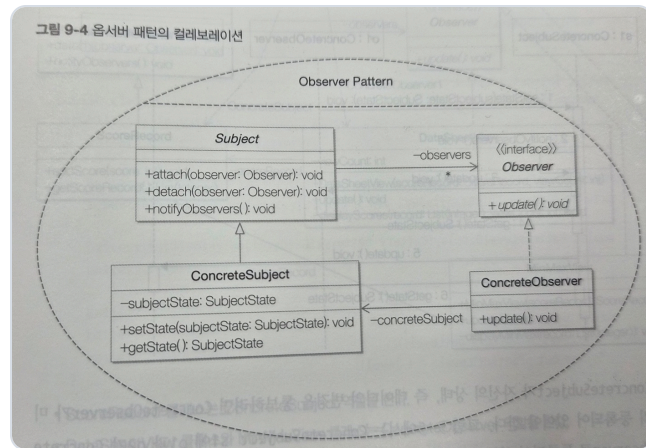
```
// 원래 RefundHandler 의 if/else 였던 '선택' 책임을 추출(SRP).
public RefundPolicy resolve(FareRule fareRule) {
    if (fareRule == null || !fareRule.isRefundable()) return new NoRefundPolicy();
    String fc = fareRule.getFareClass();
    if ("Y".equals(fc) || "B".equals(fc)) return new FullRefundPolicy();
    return new PartialRefundPolicy(); // 새 정책 추가 시 여기만 확장(OCP)
}
```




DP#3 Observer

이벤트 발생원과 반응의 분리 (pull)

교과서



Subject=EventPublisher, **Observer**=EventListener. ConcreteSubject 4 + ConcreteObserver 4.

- pull — 무인자 notifyObservers / update 후 subject.getState

구현 다이어그램

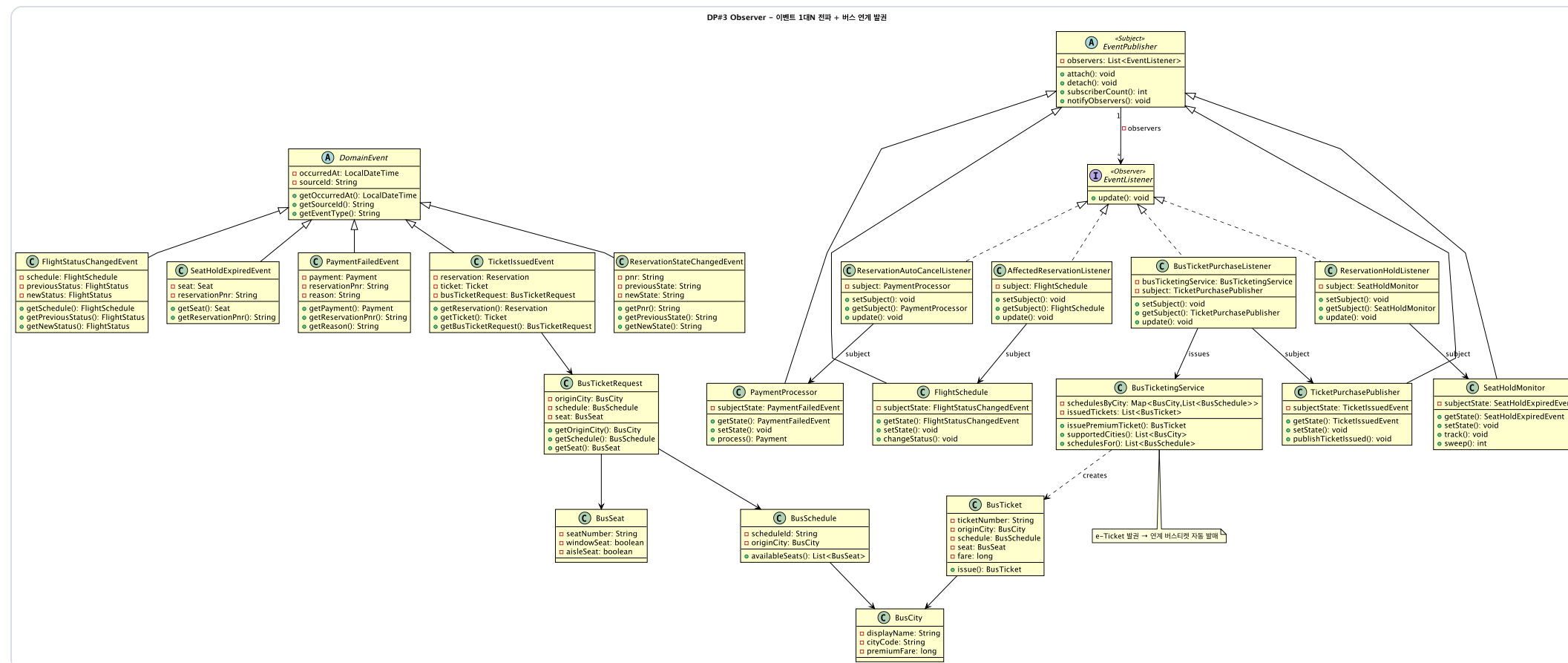
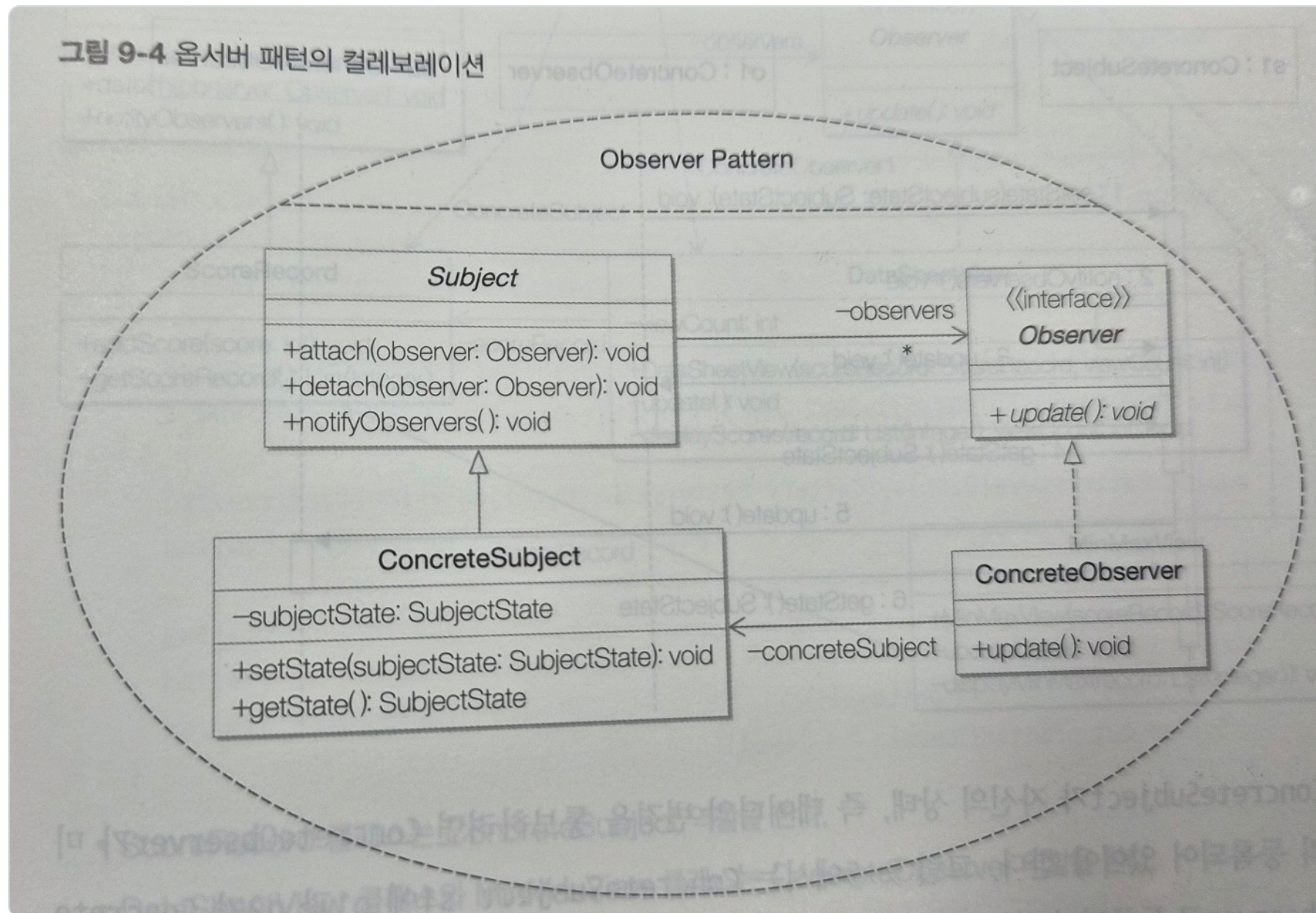
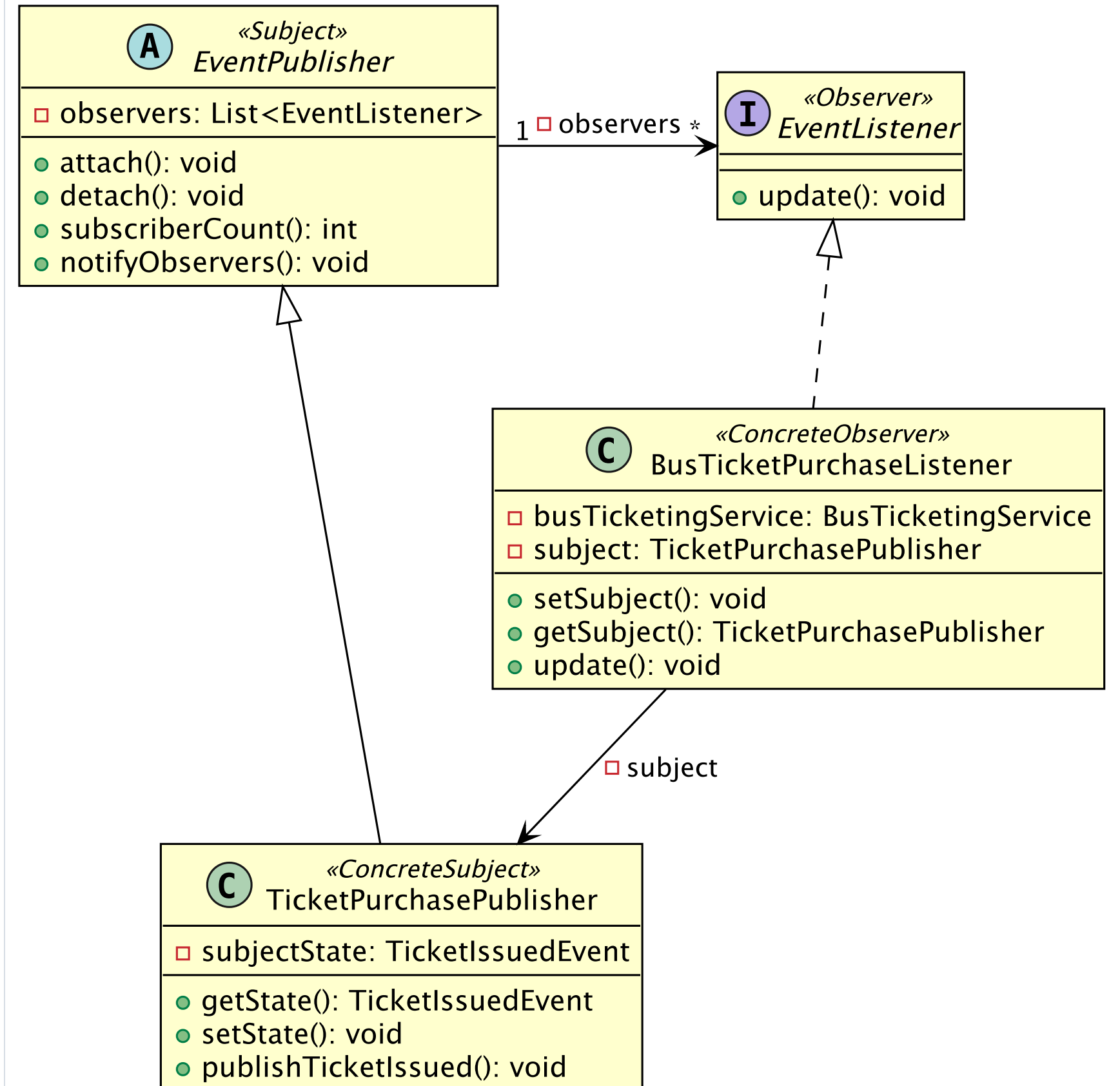


그림 9-4 옵저버 패턴의 컬레보레이션



DP#3 Observer (교과서 대응 · 그림 9-4)



DP#3 Observer — 코드 (관련 클래스 전부)

Subject=EventPublisher, Observer=EventListener. ConcreteSubject 4 + ConcreteObserver 4.

EventListener.java

Observer · 반응 인터페이스

```
// 통지를 받는 쪽의 공통 타입. 인자 없는 update() 하나뿐(pull 방식).
public interface EventListener {
    void update();
}
```

EventPublisher.java

Subject · 발생원 기반 클래스

```
private final List<EventListener> observers = new ArrayList<>();
public void attach(EventListener o) {           // 구독 등록
    if (o != null && !observers.contains(o)) observers.add(o);
}
// push 아님 - 인자 없이 update()만 호출, 상태는 옵서버가 직접 pull
public void notifyObservers() {
    for (EventListener o : new ArrayList<>(observers)) {
        try { o.update(); }                      // 한 옵서버 예외가 전체를 막지 않음
        catch (RuntimeException e) { /* 로그만 */ }
    }
}
```

BusTicketPurchaseListener.java

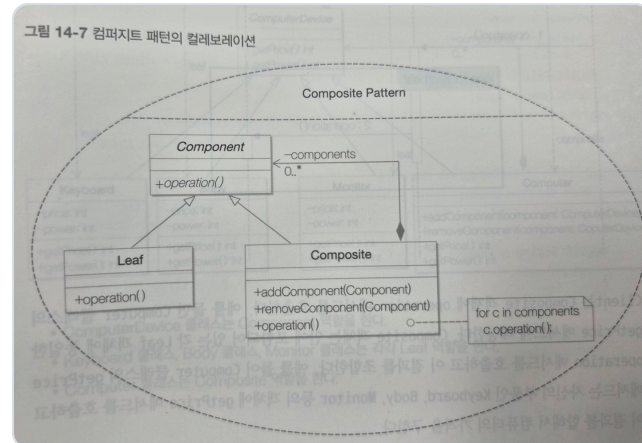
ConcreteObserver · 교수님 지시로 추가

```
public class BusTicketPurchaseListener implements EventListener {
    private TicketPurchasePublisher subject;
    public void update() {                          // 통지 수신
        TicketIssuedEvent ev = subject != null ? subject.getState() : null; // 상태 pull
        if (ev == null) return;
        BusTicketRequest req = ev.getBusTicketRequest();
        if (req == null || req.getOriginCity() == null) return;
        // 항공 발권 완료 이벤트 → 연계 버스 티켓 자동 발권
        busTicketingService.issuePremiumTicket(ev.getReservation(),
            ev.getTicket(), req.getOriginCity(), req.getSchedule(), req.getSeat());
    }
}
```

DP#4 Composite

공항과 다공항 도시를 동일 취급

교과서



Component=AirportLocation, **Leaf**=Airport, **Composite**=AirportCity.

- AirportCity.getAirports가 자식을 재귀 순회
- 검색 코드가 대상 타입으로 분기 안 함

구현 다이어그램

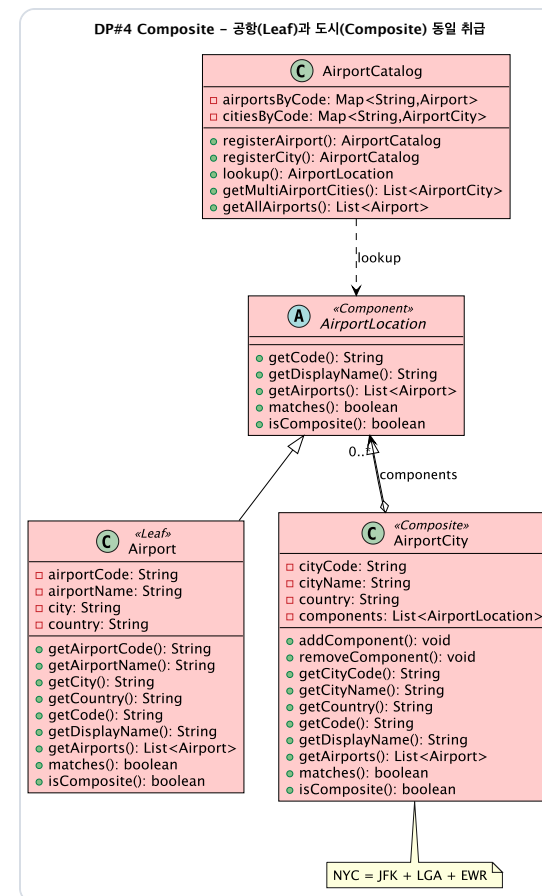
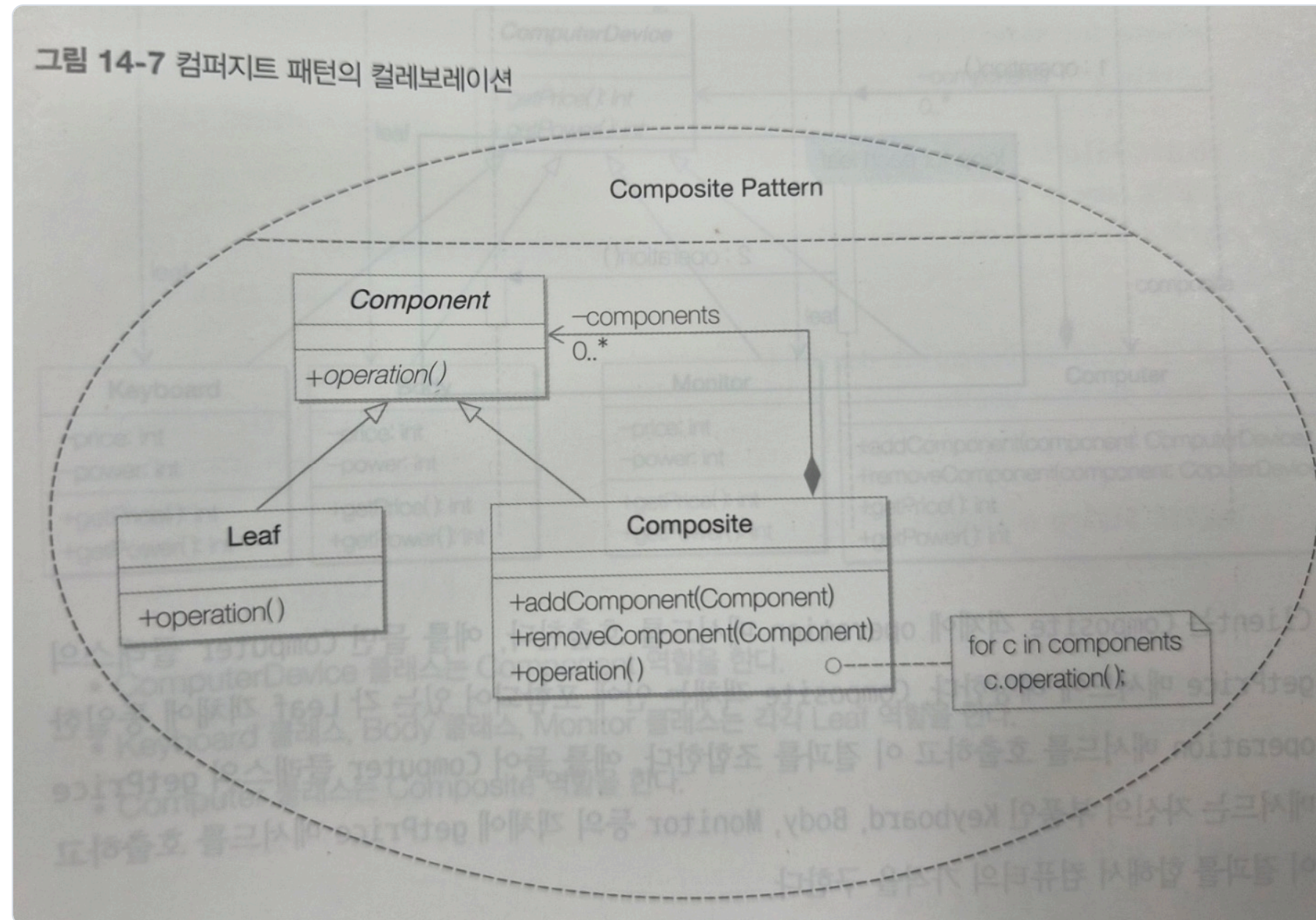
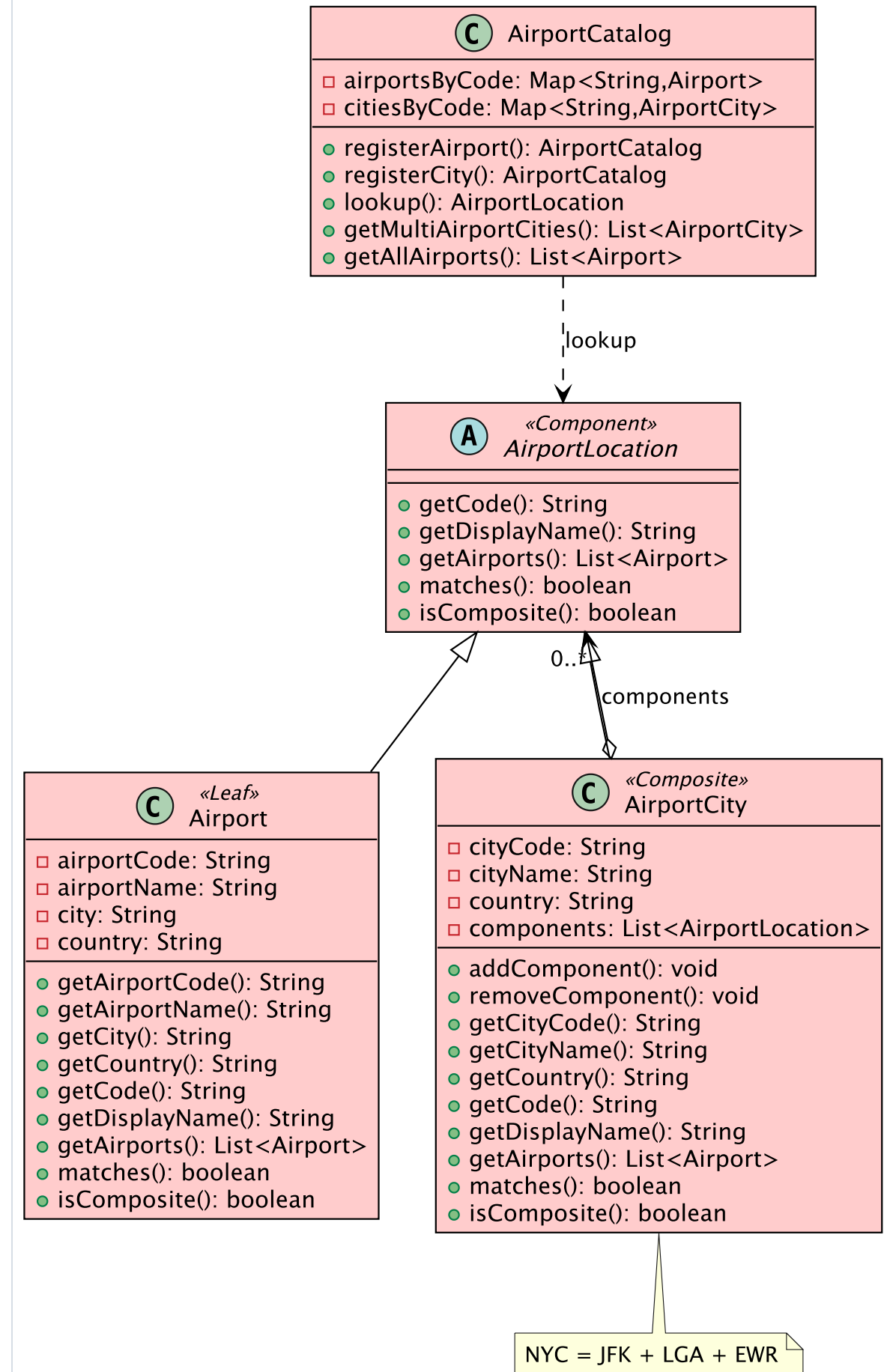


그림 14-7 컴퍼지트 패턴의 컬레보레이션



DP#4 Composite - 공항(Leaf)과 도시(Composite) 동일 취급



🌳 DP#4 Composite — 코드 (관련 클래스 전부)

Component=AirportLocation, Leaf=Airport, Composite=AirportCity.

AirportLocation.java

Component · 공통 추상 타입

```
// Leaf(Airport)와 Composite(AirportCity)가 똑같이 구현.  
// 클라이언트는 단일 공항인지 도시인지 구분 없이 이 타입만 다룬다.  
public abstract class AirportLocation {  
    public abstract String getCode();  
    public abstract List<Airport> getAirports(); // 핵심: 항상 공항 목록 반환  
    public abstract boolean matches(String code);  
    public abstract boolean isComposite();  
}
```

Airport.java

Leaf · 말단(단일 공항)

```
public class Airport extends AirportLocation {  
    // Leaf 의 getAirports()는 자기 자신 하나만 담아 반환 — 재귀 종료점  
    public List<Airport> getAirports() { return List.of(this); }  
    public boolean matches(String code) {  
        return airportCode != null && airportCode.equalsIgnoreCase(code);  
    }  
    public boolean isComposite() { return false; }  
}
```

AirportCity.java

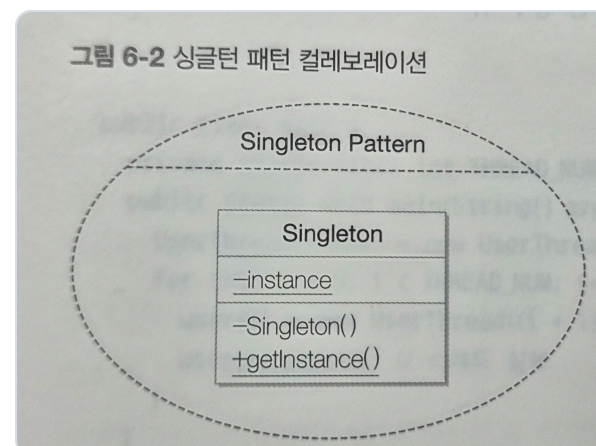
Composite · 다공항 도시

```
public class AirportCity extends AirportLocation {  
    private final List<AirportLocation> components = new ArrayList<>(); // 자식들  
    public void addComponent(AirportLocation c) { components.add(c); }  
    // 자식 전체를 재귀적으로 평탄화 — Leaf는 자기 자신, 중첩 도시는 그 자식 전부  
    public List<Airport> getAirports() {  
        List<Airport> all = new ArrayList<>();  
        for (AirportLocation c : components) all.addAll(c.getAirports()); // 재귀  
        return Collections.unmodifiableList(all);  
    }  
    public boolean isComposite() { return true; }  
}
```


DP#5 Singleton

하나의 공유 전역 설정

교과서



AppConfig — private 생성자, static getInstance.

- volatile 인스턴스에 double-checked locking
- 모든 화면이 단일 권위 설정을 읽음

구현 다이어그램

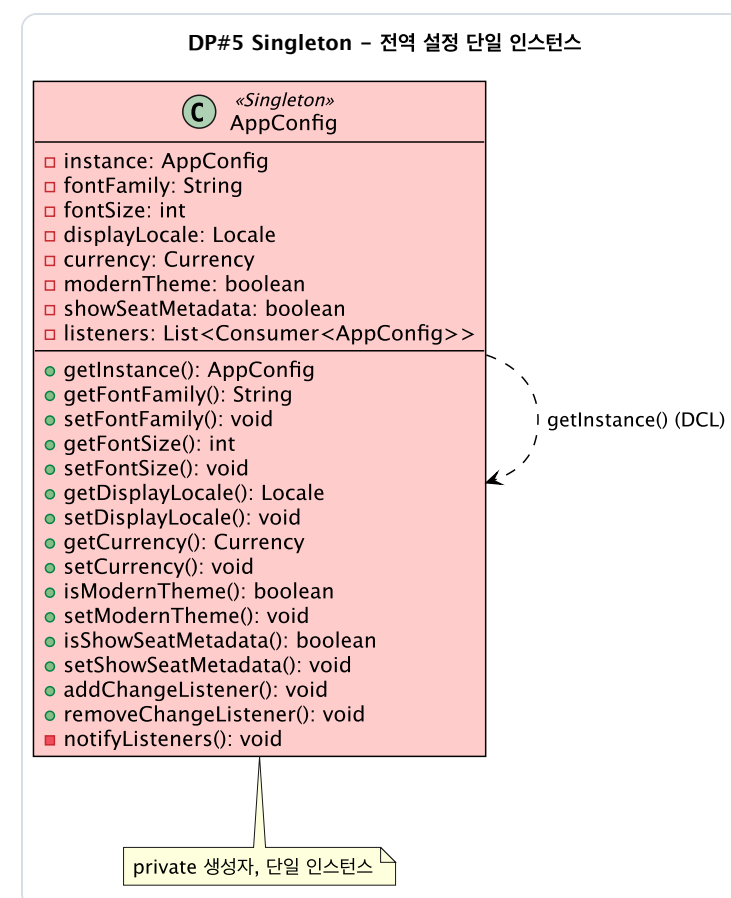


그림 6-2 싱글턴 패턴 컬레보레이션

Singleton Pattern

Singleton

-instance

-Singleton()

+getInstance()

DP#5 Singleton - 전역 설정 단일 인스턴스

Ⓒ «Singleton»
AppConfig

- instance: AppConfig
- fontFamily: String
- fontSize: int
- displayLocale: Locale
- currency: Currency
- modernTheme: boolean
- showSeatMetadata: boolean
- listeners: List<Consumer<AppConfig>>

- getInstance(): AppConfig
- getFontFamily(): String
- setFontFamily(): void
- getFontSize(): int
- setFontSize(): void
- getDisplayLocale(): Locale
- setDisplayLocale(): void
- getCurrency(): Currency
- setCurrency(): void
- isModernTheme(): boolean
- setModernTheme(): void
- isShowSeatMetadata(): boolean
- setShowSeatMetadata(): void
- addChangeListener(): void
- removeChangeListener(): void
- notifyListeners(): void

getInstance() (DCL)

private 생성자, 단일 인스턴스

DP#5 Singleton — 코드 (관련 클래스 전부)

AppConfig — private 생성자, static getInstance.

AppConfig.java

Singleton · 유일 인스턴스 보장

```
public final class AppConfig {
    private static volatile AppConfig instance;    // volatile — 가시성 보장
    private AppConfig() { }                      // private 생성자 — 외부 new 차단
    public static AppConfig getInstance() {        // double-checked locking
        AppConfig local = instance;
        if (local == null) {
            synchronized (AppConfig.class) {
                local = instance;
                if (local == null) instance = local = new AppConfig();
            }
        }
        return local;
    }
}
```

AppConfig.java

공유 전역 상태 + 변경 통지

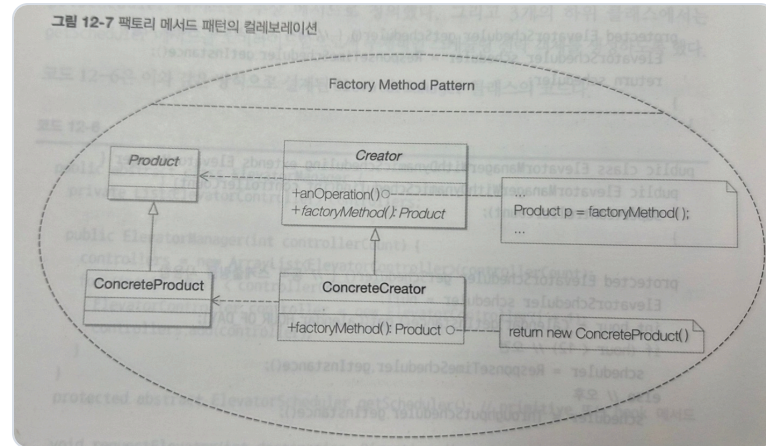
```
private String fontFamily = "Pretendard";
private Locale displayLocale = Locale.KOREA;
private Currency currency = Currency.getInstance("KRW");
// 모든 화면이 같은 인스턴스를 읽으므로 설정이 한 곳에서 권위를 가진다.
public void setFontFamily(String f) {
    this.fontFamily = f;
    notifyListeners();                // 구독 화면들에 갱신 통지
}
```



DP#6 Factory Method

각 종류가 자기 제품을 생성

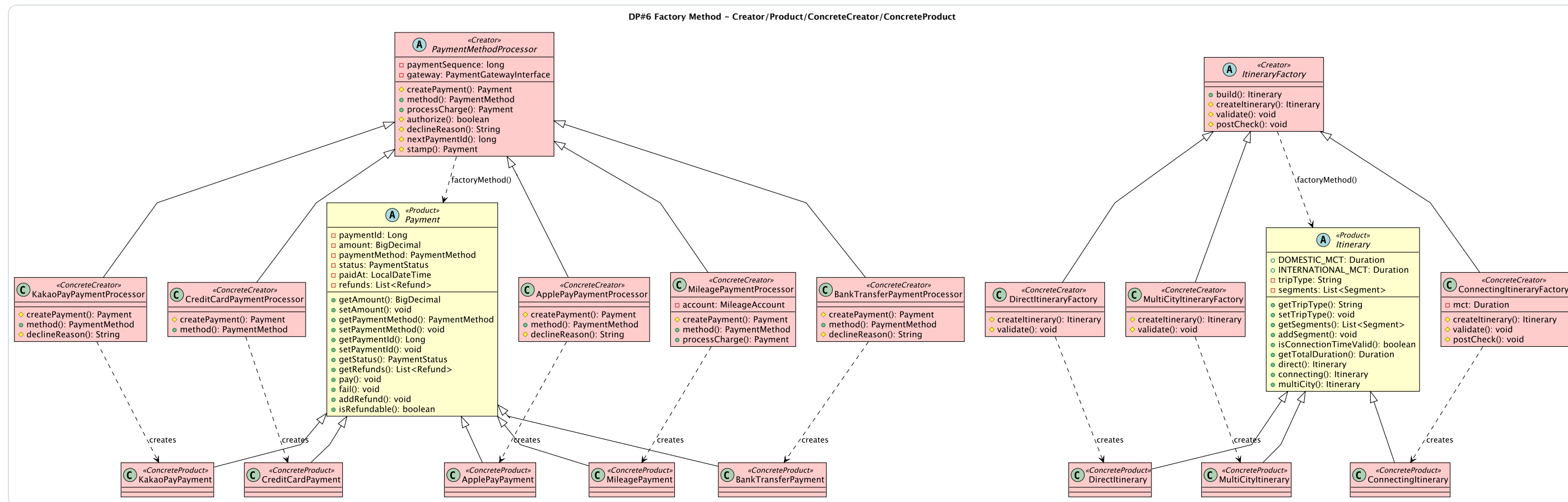
교과서

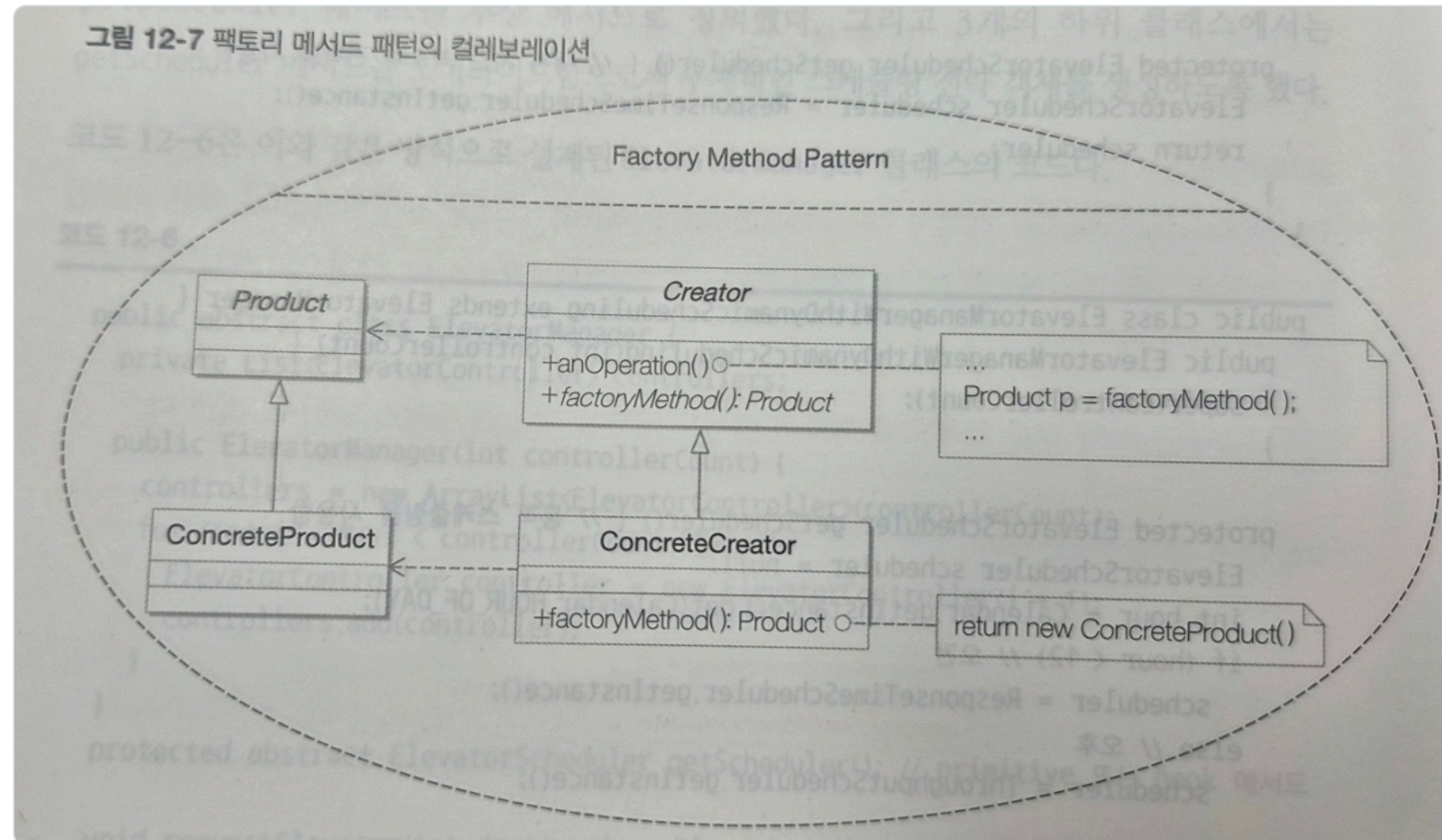


Creator=PaymentMethodProcessor, **Product**=Payment.

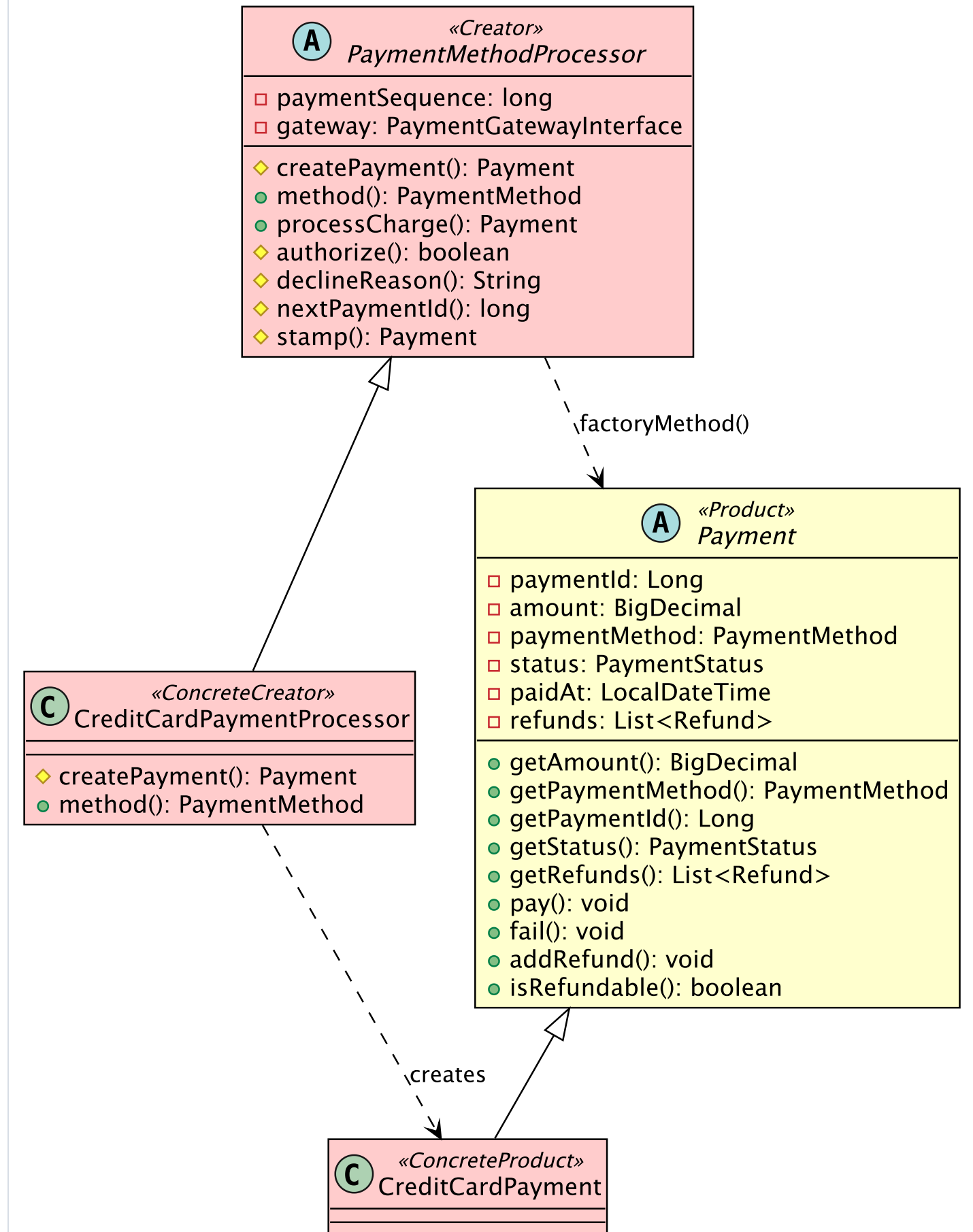
- 5개 ConcreteCreator가 무인자 createPayment override
- ItineraryFactory도 동일 구조
- 생성된 Payment 승인은 외부 PG **legacy system**을 PaymentGatewayInterface로 격리 (MockPaymentGateway로 대체)

구현 다이어그램





DP#6 Factory Method (교과서 대응 · 그림 12-7)



DP#6 Factory Method — 코드 (관련 클래스 전부)

Creator=PaymentMethodProcessor, Product=Payment.

PaymentMethodProcessor.java

Creator · 템플릿 + 팩토리 메서드

```
public abstract class PaymentMethodProcessor {
    public abstract Payment createPayment();          // 팩토리 메서드(무인자)
    public abstract PaymentMethod method();
    // anOperation: 제품 '생성'은 서브클래스에 위임, 공통 흐름은 여기서 고정
    public Payment processCharge(long amount, String pnr) {
        Payment payment = stamp(createPayment(), amount); // createPayment()로 생성
        if (authorize(payment)) payment.pay(); else payment.fail();
        return payment;
    }
}
```

CreditCardPaymentProcessor.java

ConcreteCreator · 5종 중 1

```
public class CreditCardPaymentProcessor extends PaymentMethodProcessor {
    public Payment createPayment() { return new CreditCardPayment(); } // 자기 제품
    public PaymentMethod method() { return PaymentMethod.CREDIT_CARD; }
}
// ApplePay / KakaoPay / BankTransfer / Mileage 도 동일 - 새 수단 = ConcreteCreator 추가(OCP)
```

Payment.java

Product · 제품 추상 타입

```
public abstract class Payment {
    private BigDecimal amount;
    private PaymentStatus status;
    protected Payment(PaymentMethod method) { this.paymentMethod = method; }
    public void pay() { /* 상태 전이 → PAID */ }
    public void fail() { /* 상태 전이 → FAILED */ }
}
```

CreditCardPayment.java

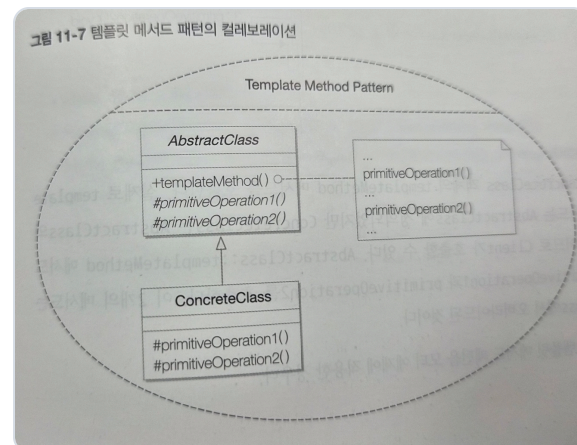
ConcreteProduct

```
public class CreditCardPayment extends Payment {
    public CreditCardPayment() { super(PaymentMethod.CREDIT_CARD); }
}
```

DP#7 Template Method

티켓 렌더 골격 고정

교과서



AbstractClass=TicketRenderer, final render().

- 추상 header / body / footer, hook separator
- Plain / HTML / BoardingPass가 단계 override

구현 다이어그램

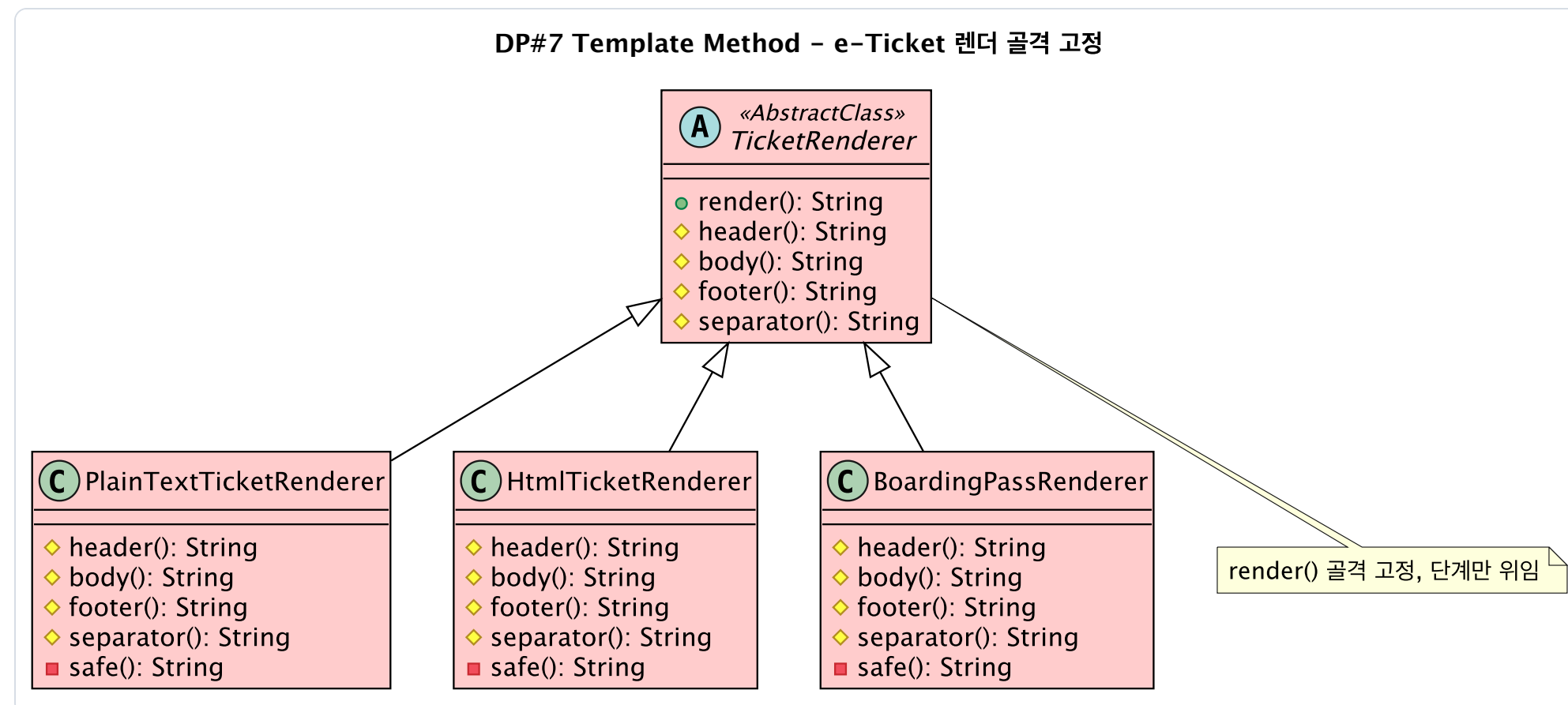
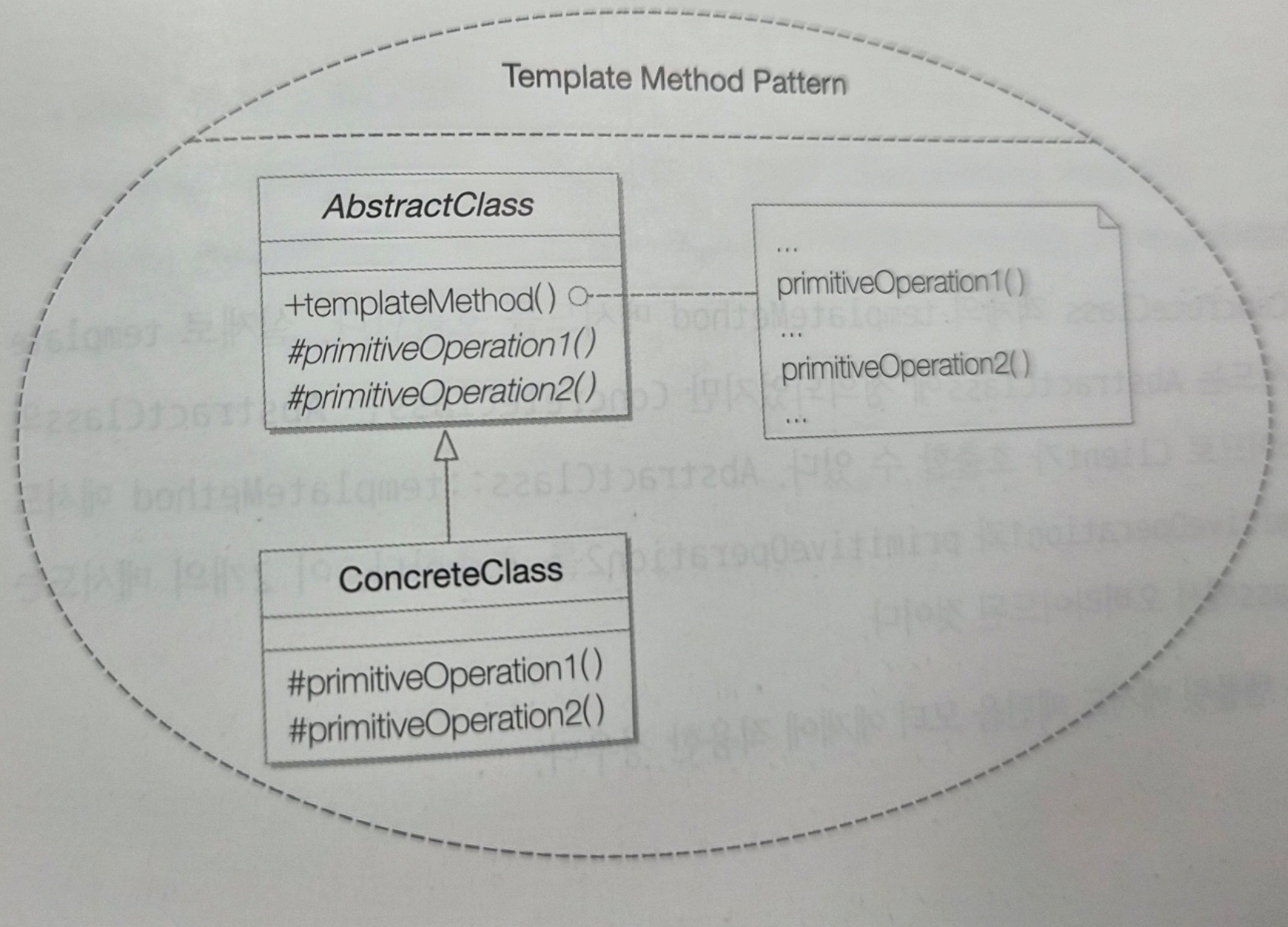
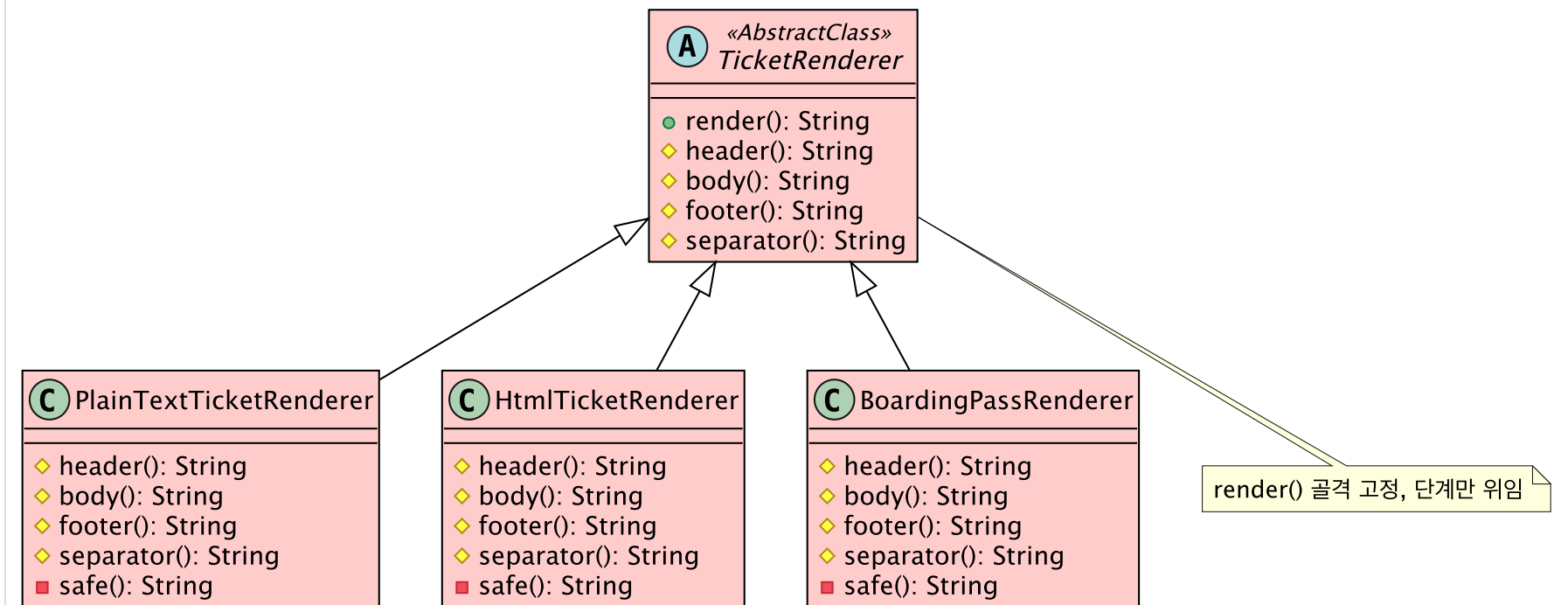


그림 11-7 템플릿 메서드 패턴의 컬레보레이션



DP#7 Template Method - e-Ticket 렌더 골격 고정





DP#7 Template Method — 코드 (관련 클래스 전부)

AbstractClass=TicketRenderer, final render().

TicketRenderer.java

AbstractClass · 골격 고정(final)

```
public abstract class TicketRenderer {
    // 템플릿 메서드 — final 로 호출 순서 고정, 서브클래스가 못 바꿈
    public final String render(Reservation r, Ticket t) {
        StringBuilder sb = new StringBuilder();
        sb.append(header(r, t)).append(separator())
          .append(body(r, t)).append(separator()).append(footer(r, t));
        return sb.toString();
    }
    protected abstract String header(Reservation r, Ticket t); // primitive 단계
    protected abstract String body(Reservation r, Ticket t);
    protected abstract String footer(Reservation r, Ticket t);
    protected String separator() { return "\n"; } // hook — override 선택
}
```

PlainTextTicketRenderer.java

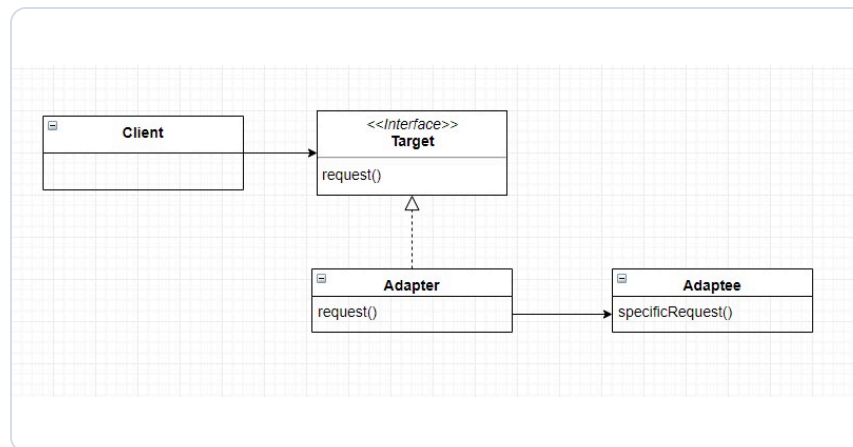
ConcreteClass · 단계만 구현

```
public class PlainTextTicketRenderer extends TicketRenderer {
    protected String header(Reservation r, Ticket t) {
        return "=== KOREAN AIR e-TICKET ===" + safe(t.getTicketNumber());
    }
    protected String body(Reservation r, Ticket t) { /* 승객·발권시각 */ }
    protected String footer(Reservation r, Ticket t) { /* 게이트 안내 문구 */ }
    protected String separator() { return "-----"; } // hook 재정의
}
// HTML / BoardingPass 렌더러도 동일하게 단계만 교체
```


DP#8 Adapter

외부 Skypass 마일리지 legacy system 감싸기

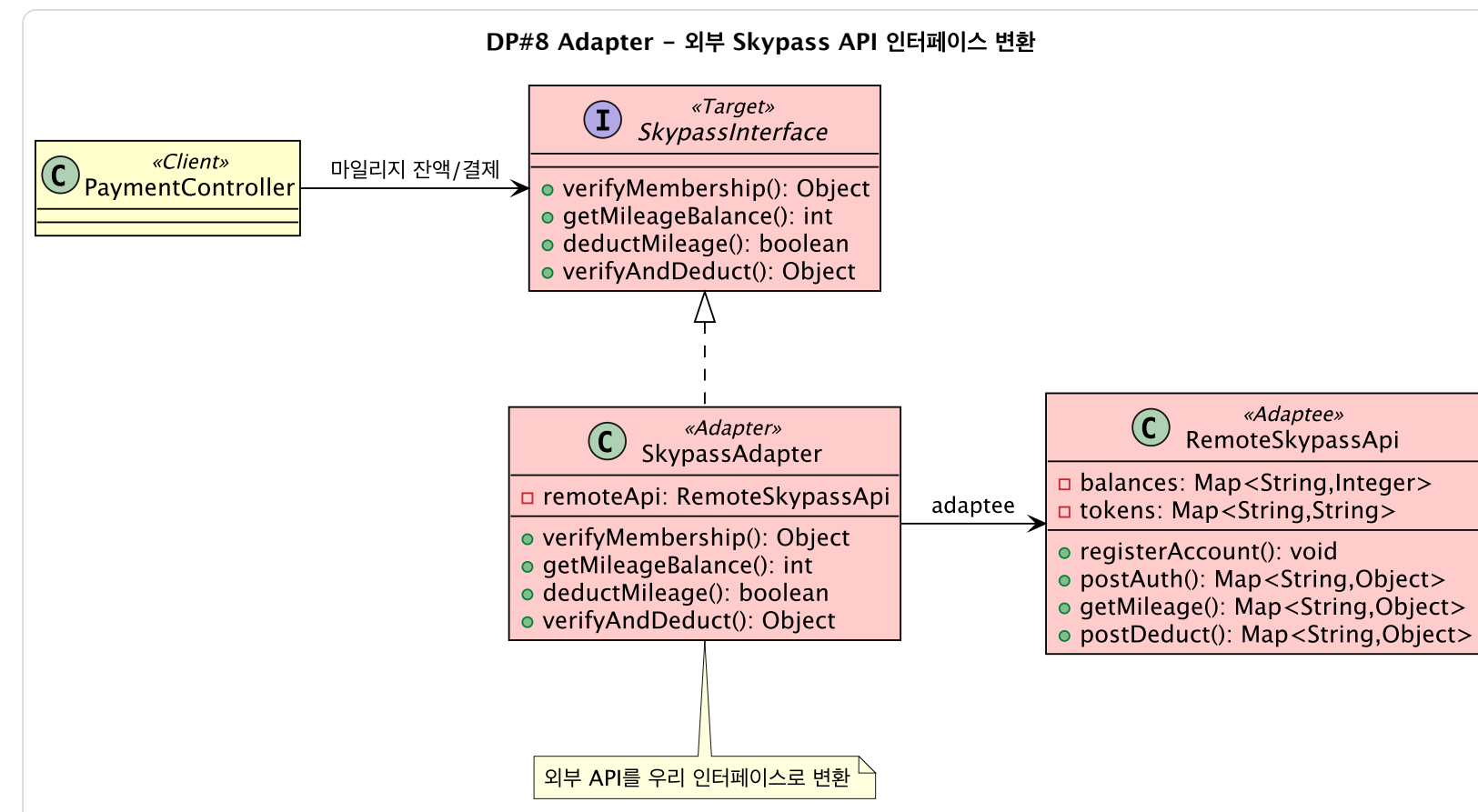
교과서

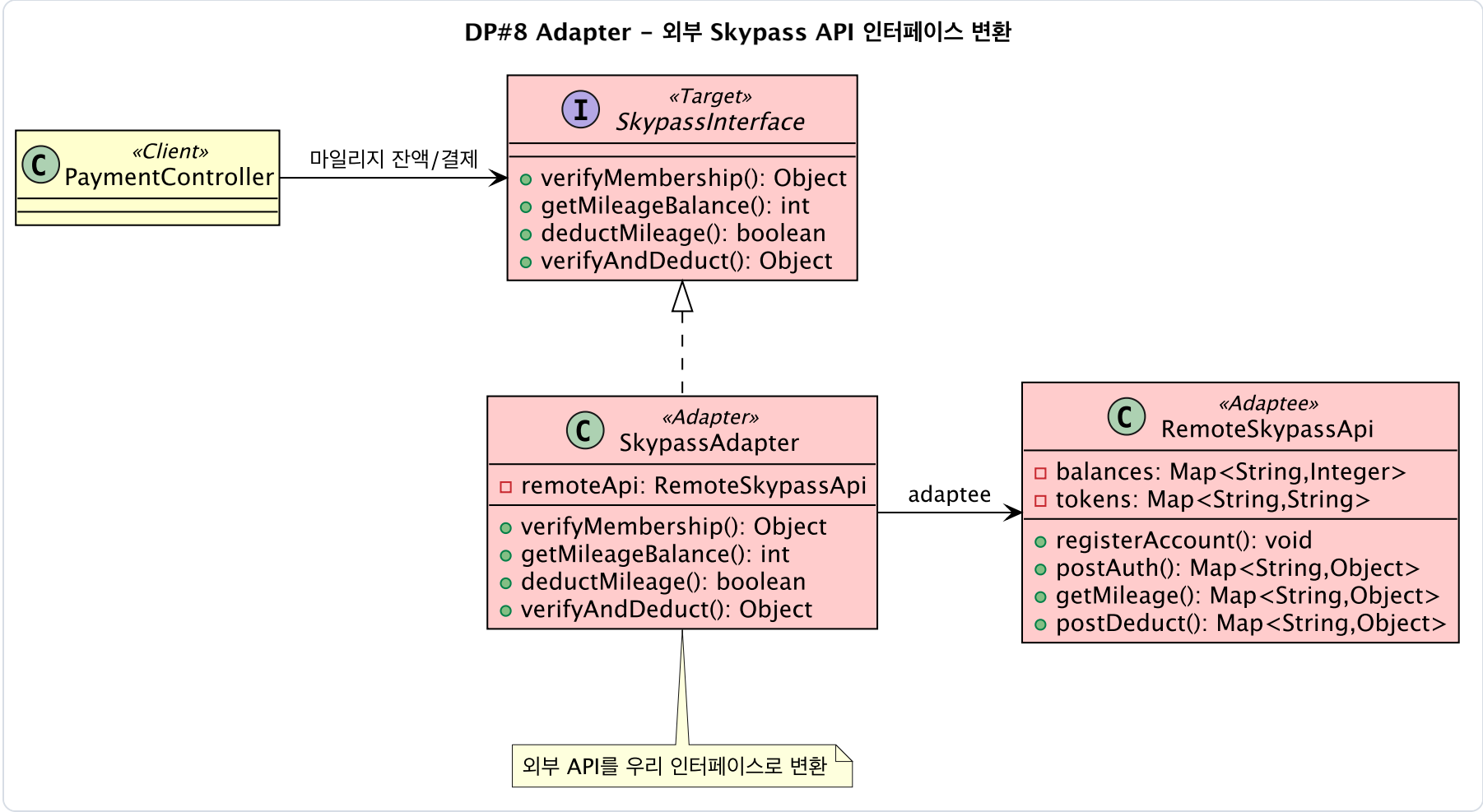
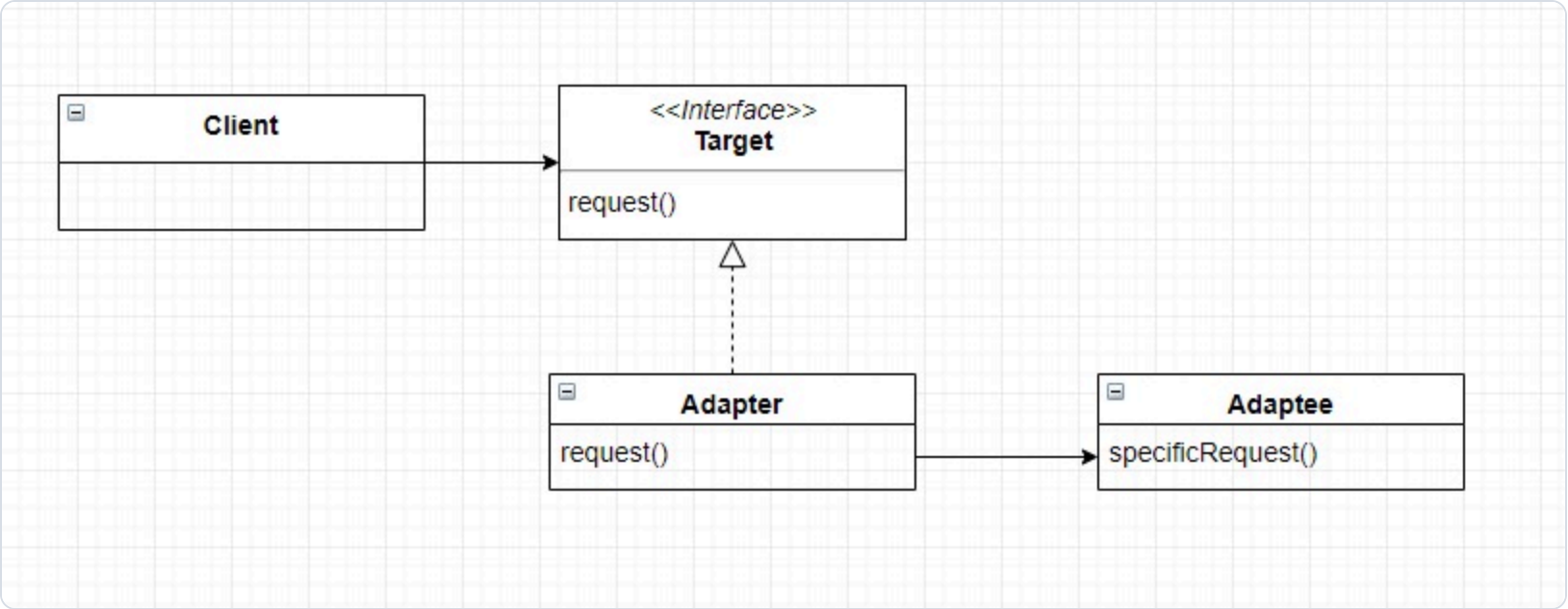


Target=SkypassInterface, **Adapter**=SkypassAdapter, **Adaptee**=RemoteSkypassApi.

- 수정할 수 없는 legacy system(외부 회원사 API)을 우리 코드 변경 없이 통합
- deductMileage가 adaptee.postDeduct Map을 boolean으로 변환
- 애플리케이션은 우리 인터페이스에만 의존 (DIP)

구현 다이어그램





🔌 DP#8 Adapter — 코드 (관련 클래스 전부)

Target=SkypassInterface, Adapter=SkypassAdapter, Adaptee=RemoteSkypassApi.

SkypassInterface.java

Target · 우리가 원하는 인터페이스

```
// 애플리케이션이 의존하는 도메인 친화적 타입(boolean / int 반환).
public interface SkypassInterface {
    boolean deductMileage(String skypassNumber, int amount);
    int getMileageBalance(String skypassNumber);
}
```

SkypassAdapter.java

Adapter · 변환기

```
public class SkypassAdapter implements SkypassInterface {
    private final RemoteSkypassApi adaptee; // legacy 외부 시스템을 보유
    // 외부 Map 응답을 도메인 boolean 으로 변환 — 호출부는 Map 을 몰라도 됨
    public boolean deductMileage(String no, int amount) {
        Map<String, Object> res = adaptee.postDeduct(no, amount);
        return Boolean.TRUE.equals(res.get("success"));
    }
}
```

RemoteSkypassApi.java

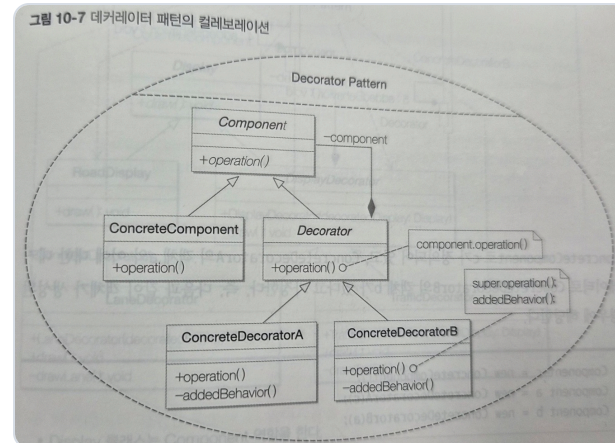
Adaptee · legacy 외부 시스템(수정 불가)

```
// legacy 외부 회사 API — 시그니처가 우리와 다름(Map 반환). 수정 불가 가정.
public class RemoteSkypassApi {
    public Map<String, Object> postDeduct(String memberCode, int amount) {
        Map<String, Object> res = new HashMap<>();
        // ... 잔액 차감 후 success / remaining 키를 담은 Map 반환
        res.put("success", true);
        res.put("remaining", remaining);
        return res;
    }
}
```

DP#9 Decorator

런타임에 좌석 옵션 누적

교과서



Component=SeatView, **Decorator**=AbstractSeatDecorator.

- ExtraLegroom(+50,000), Lounge(+80,000)는 super에 더해 요금 / Window, Aisle은 라벨만
- 조합마다 클래스 안 만들고 옵션 결합 (OCP)

구현 다이어그램

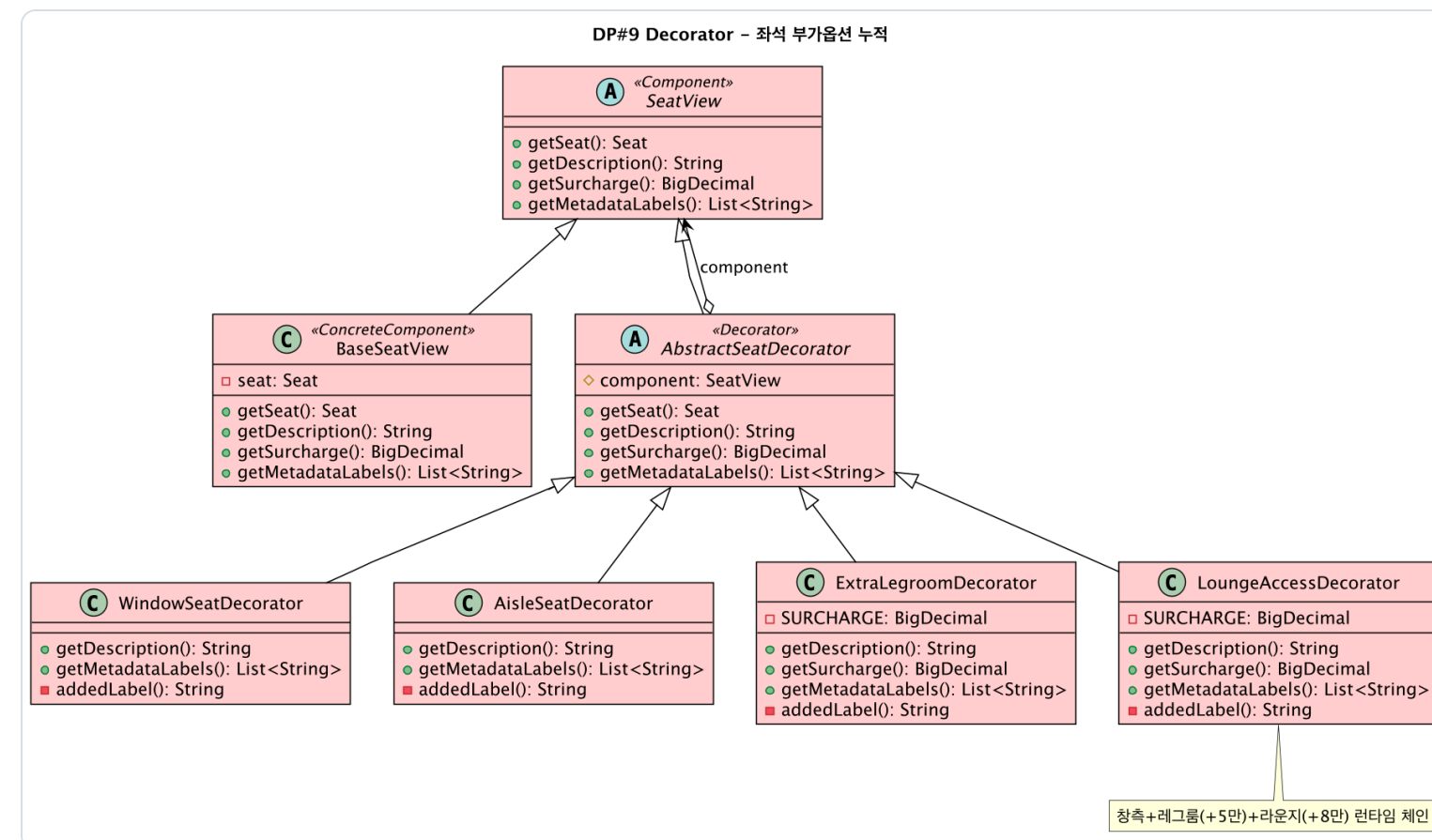
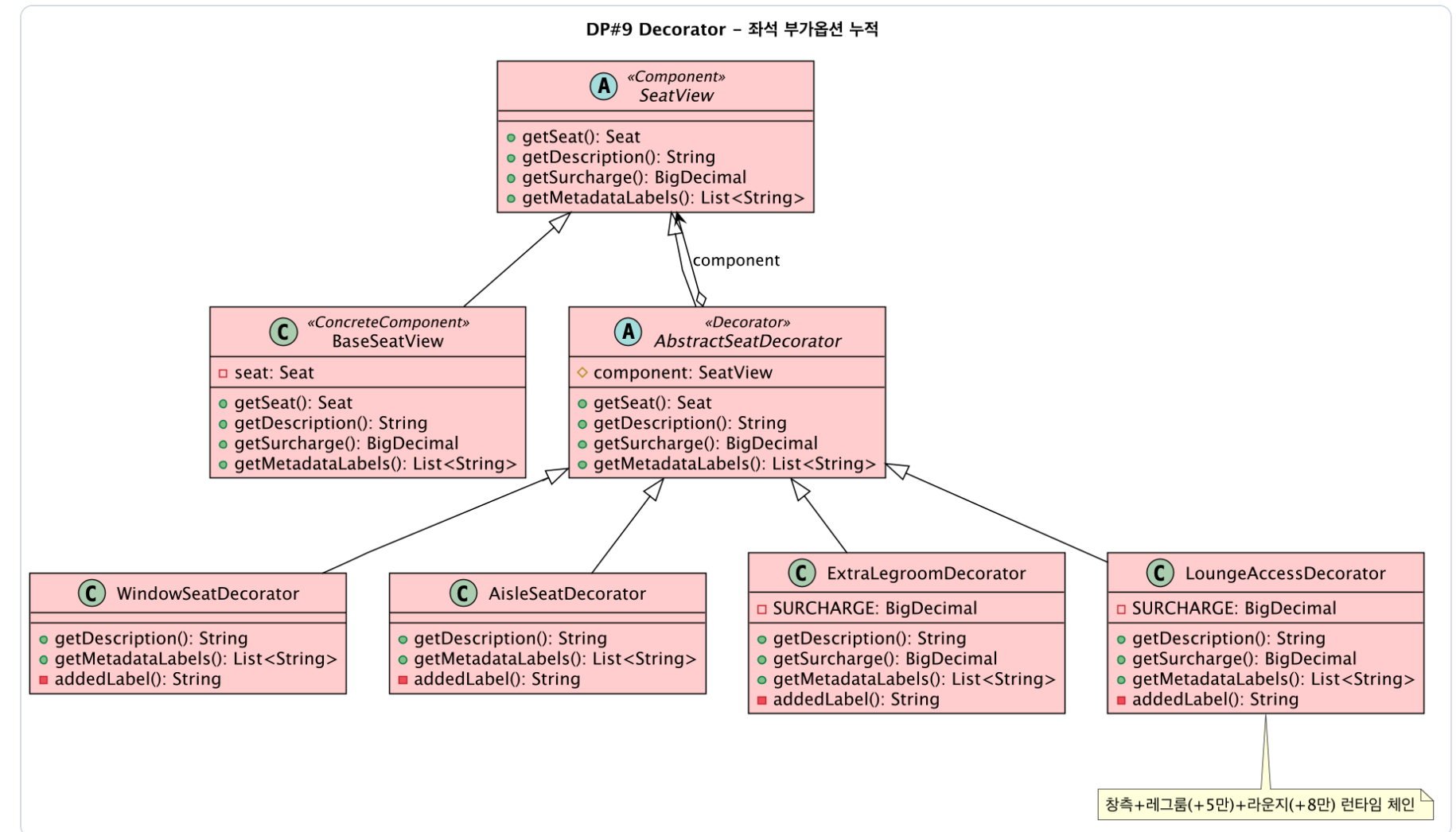
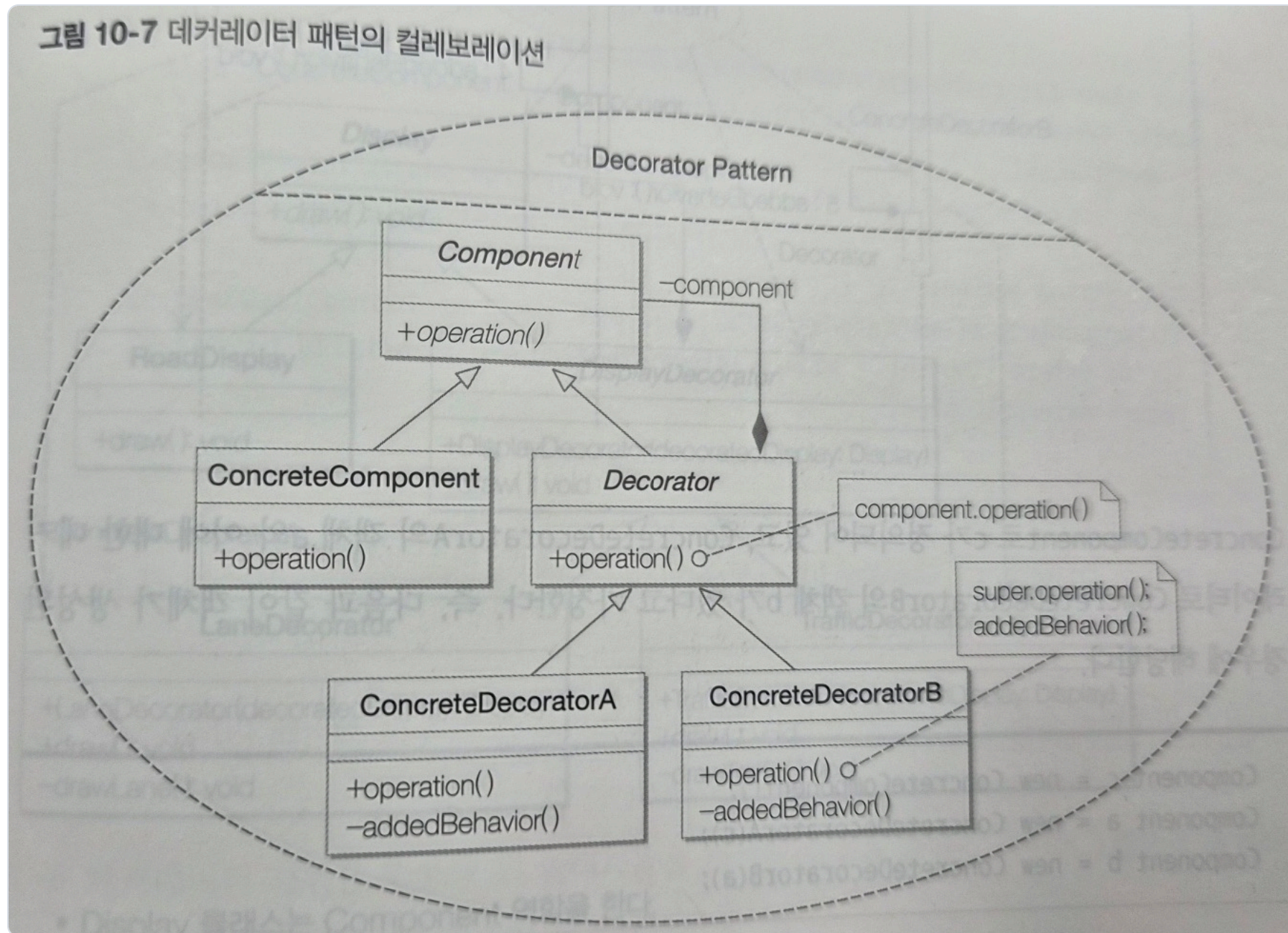


그림 10-7 데커레이터 패턴의 컬레보레이션



DP#9 Decorator — 코드 (관련 클래스 전부)

Component=SeatView, Decorator=AbstractSeatDecorator.

SeatView.java

Component · 공통 추상 타입

```
// 기본 좌석과 데코레이터가 똑같이 구현하는 타입.
public abstract class SeatView {
    public abstract BigDecimal getSurcharge();    // 추가 요금
    public abstract String getDescription();
    public abstract List<String> getMetadataLabels();
}
```

AbstractSeatDecorator.java

Decorator · 감싼 객체로 위임

```
public abstract class AbstractSeatDecorator extends SeatView {
    protected final SeatView component;          // 감싼 대상(같은 타입)
    protected AbstractSeatDecorator(SeatView c) { this.component = c; }
    // 기본 동작은 그대로 위임 — 서브클래스가 필요한 메서드만 덧붙인다
    public BigDecimal getSurcharge() { return component.getSurcharge(); }
    public String getDescription() { return component.getDescription(); }
}
```

ExtraLegroomDecorator.java

ConcreteDecorator · 요금 누적

```
public class ExtraLegroomDecorator extends AbstractSeatDecorator {
    private static final BigDecimal SURCHARGE = new BigDecimal("50000");
    public BigDecimal getSurcharge() {
        return super.getSurcharge().add(SURCHARGE);    // 감싼 결과 + 자기 요금(+50,000)
    }
    public String getDescription() {
        return super.getDescription() + " · 프리미엄 레그룸 (+50,000)";
    }
}
// Lounge(+80,000) 등도 동일 — 조합마다 클래스 안 만들고 런타임에 겹쳐 씬(OCF)
```

🚩 디자인 패턴 적용 강도

아홉 패턴(State 포함)이 하나의 실행 가능한 흐름에서 함께 동작한다

기능 완결도

아홉 패턴 모두 end-to-end 흐름에서 실제 동작.

예: 결제는 Factory Method, 환불은 Strategy, 발권 통지는 Observer, 좌석은 Decorator로 한 예약에서 연쇄 동작

재사용성

호출부를 고치지 않고 클래스 하나를 더해 확장.

예: 새 결제 수단 = ConcreteCreator 하나 추가, 새 알림 = Listener 하나 추가 (개방-폐쇄 원칙)

테스트 적합성

역할 경계가 또렷해 단위별 독립 검증 가능.

예: Adapter는 가짜 외부 시스템으로 교체, State는 허용되지 않는 전이를 결정적으로 거부

DP#1

State

DP#2

Strategy

DP#3

Observer

DP#4

Composite

DP#5

Singleton

DP#6

Factory Method

DP#7

Template Method

DP#8

Adapter

DP#9

Decorator

현황과 다음 단계

구현 완료

- 예약 조회, 취소 / 환불, 결제 + 마일리지
- 좌석 Decorator, e-Ticket, Observer, 상태 머신
- 주요 오류를 화면 메시지와 로그로 확인

보완 필요 · 다음

- 버스 부가 발권 실패가 화면에 미표시
- 예약 후 좌석 변경이 전방 화면 재사용
- Extract Class로 BookingController 7개 책임 완전 분리

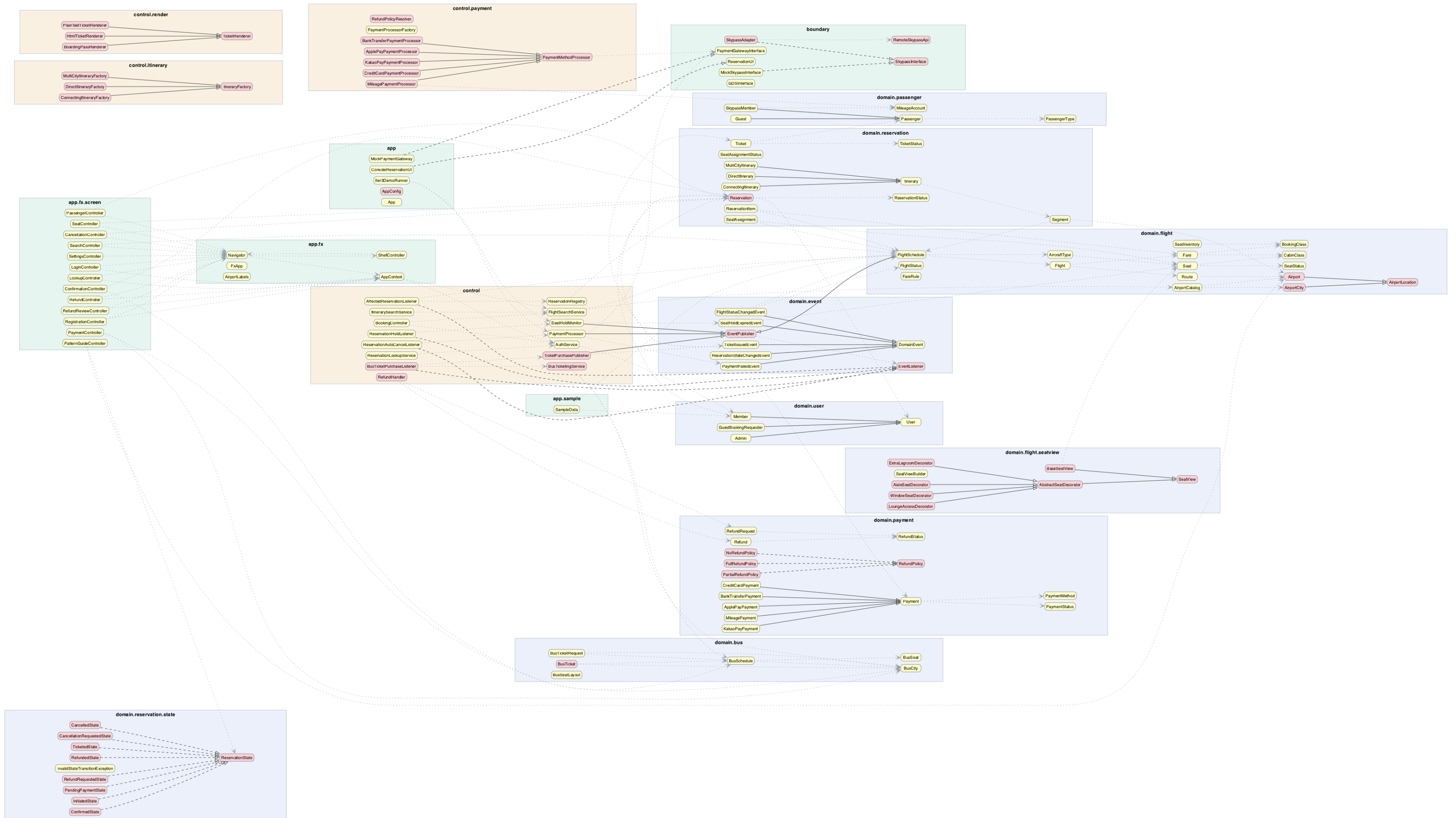
🙋 감사합니다

질문 있나요?

대한항공 예약 시스템 — OODP Iteration 4

DP#1 State … DP#9 Decorator

대한항공 예약 시스템 — 전체 클래스 다이어그램 (146 classes · ECB+9GoF · 빨강=패턴역할)



Reservation 설계 방어

"왜 Reservation 이 비대한가?" — 예상 질문 대비 backup

① 정당한 이유

Aggregate Root(예약 트리 일관성 경계) + State Context(State 구현체가 Reservation을 인자로 받음) 겸임은 DDD / GoF 정석. 데이터·행위 강제 분리는 anemic domain model 안티패턴. → God class 아님.

② 이미 한 것

영속·조회 책임을 ReservationRegistry로 Extract Class (Iteration 4). findByPnr은 호환 shim. dead 중복 위임 API·빈 stub 정리 완료.

③ 향후

status enum ↔ currentState 이중 상태 단일화, 잔여 중복 생애주기 API 정리 (저위험·선택). 공격적 전면 분해는 하지 않음.